

Chapter 2

Digital Design and Computer Architecture

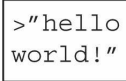
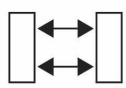
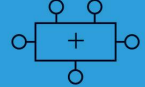
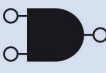
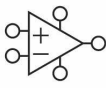


東北大學 国家示范性软件学院

SOFTWARE COLLEGE OF NORTHEASTERN UNIVERSITY

Chapter 2 :: Topics

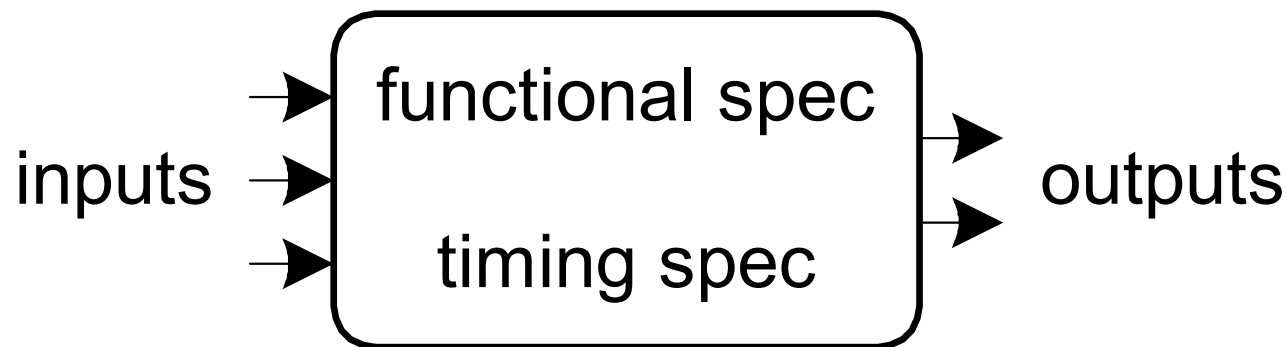
- **Introduction**
- Boolean Equations
- Boolean Algebra
- From Logic to Gates
- Combinational Logic
- X's and Z's
- Karnaugh Maps
- Combinational Building Blocks
- Timing

Application Software	
Operating Systems	
Architecture	
Micro-architecture	
Logic	
Digital Circuits	
Analog Circuits	
Devices	
Physics	

Introduction

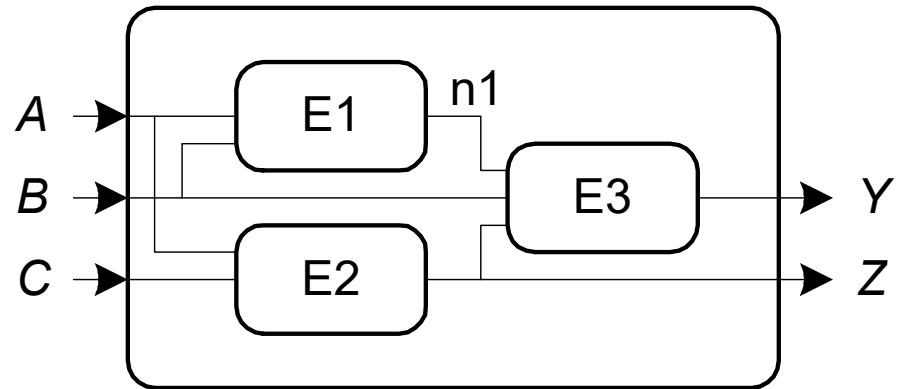
A logic circuit is composed of:

- Inputs
- Outputs
- Functional specification(规范)
- Timing specification(规范)



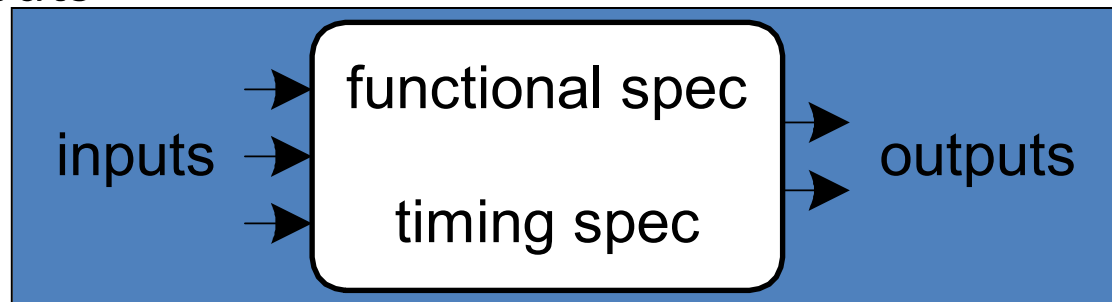
Circuits

- Nodes
 - Inputs: A, B, C
 - Outputs: Y, Z
 - Internal: $n1$
- Circuit elements
 - $E1, E2, E3$
 - Each corresponds to a circuit



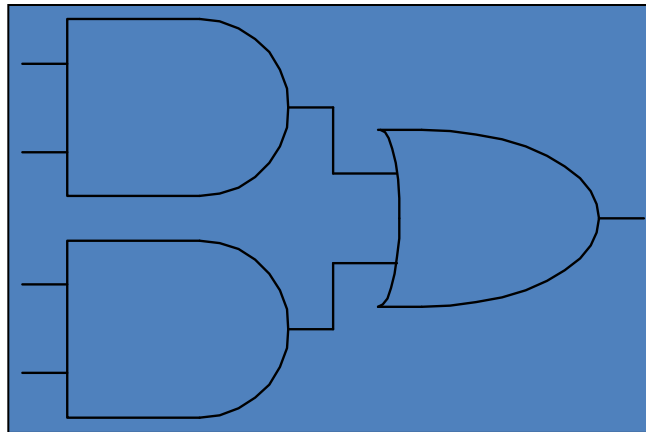
Types of Logic Circuits

- **Combinational Logic(组合逻辑)**
 - Memoryless
 - Outputs determined by current values of inputs
- **Sequential Logic(时序逻辑)**
 - Has memory
 - Outputs determined by previous and current values of inputs

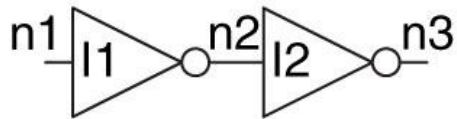


Rules of Combinational Composition

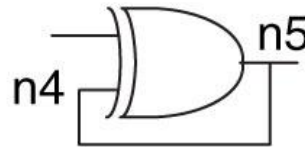
- Every element is combinational
- Every node is either an input or connects to *exactly one* output
- The circuit contains no cyclic paths
- **Example:**



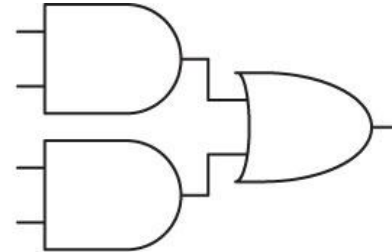
Example 2.1 COMBINATIONAL CIRCUITS



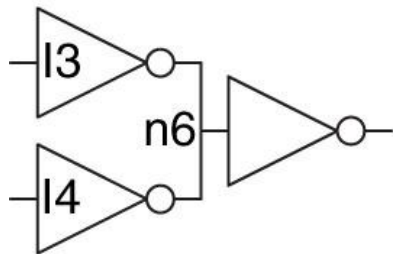
(a)



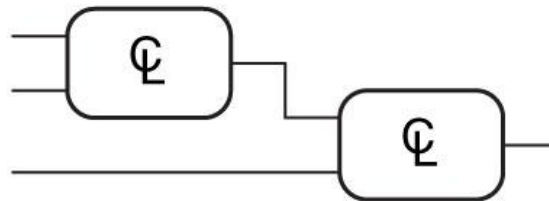
(b)



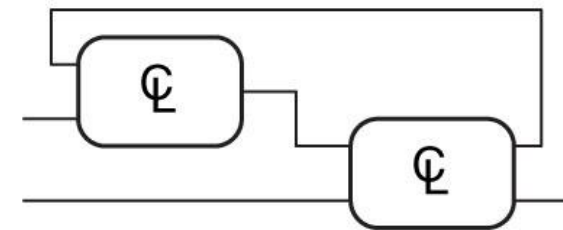
(c)



(d)



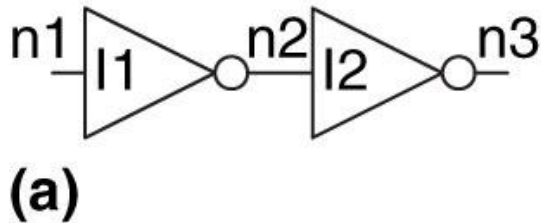
(e)



(f)

Which of the circuits are combinational circuits according to the rules of combinational composition?

Example 2.1 COMBINATIONAL CIRCUITS



(a) is combinational.

It is constructed from two combinational circuit elements (inverters I1 and I2).

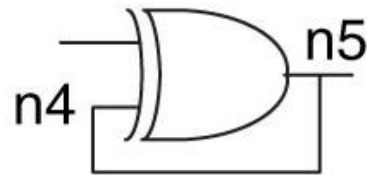
It has three nodes: n1, n2, and n3.

n1 is an input to the circuit and to I1;

n2 is an internal node, which is the output of I1 and the input to I2;

n3 is the output of the circuit and of I2.

Example 2.1 COMBINATIONAL CIRCUITS



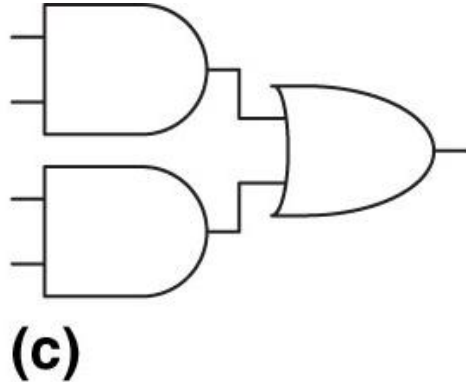
(b)

(b) is not combinational

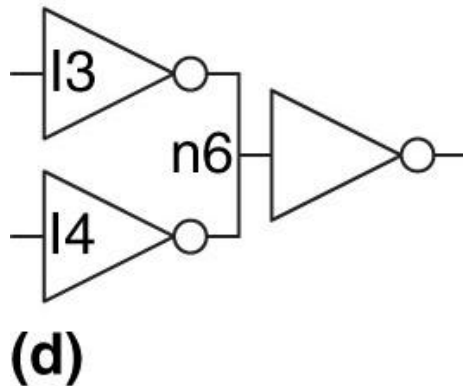
because there is a cyclic path:
the output of the XOR feeds back to one
of its inputs.

Hence, a cyclic path starting at n4 passes
through the XOR to n5, which returns to
n4.

Example 2.1 COMBINATIONAL CIRCUITS

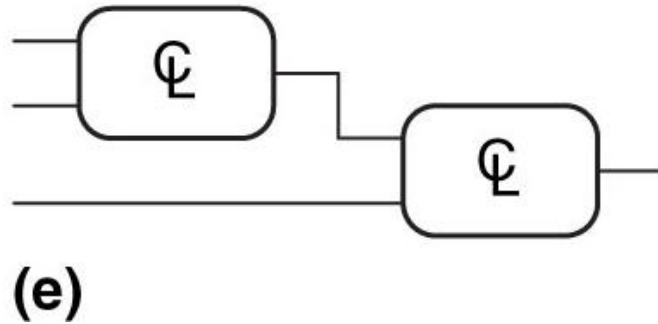


(c) is combinational.

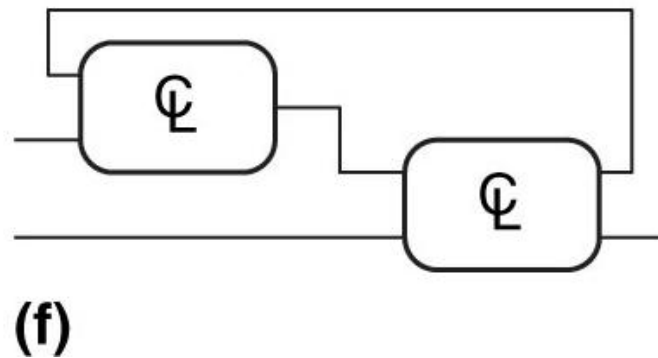


(d) is not combinational,
because node n6 connects to the output
terminals of both I3 and I4.

Example 2.1 COMBINATIONAL CIRCUITS



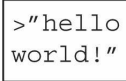
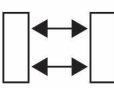
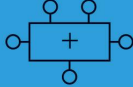
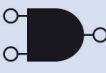
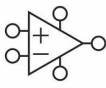
(e) is combinational, illustrating two combinational circuits connected to form a larger combinational circuit.



(f) does not obey the rules of combinational composition because it has a cyclic path through the two elements. Depending on the functions of the elements, it may or may not be a combinational circuit.

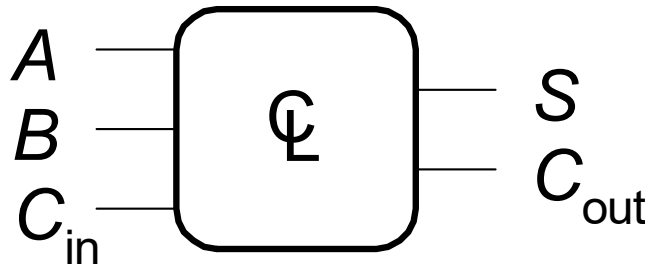
Chapter 2 :: Topics

- Introduction
- **Boolean Equations**
- Boolean Algebra
- From Logic to Gates
- X's and Z's, Oh My
- Karnaugh Maps
- Combinational Building Blocks
- Timing

Application Software	
Operating Systems	
Architecture	
Micro-architecture	
Logic	
Digital Circuits	
Analog Circuits	
Devices	
Physics	

Boolean Equations

- Functional specification of outputs in terms of inputs
- Example:** $S = F_1(A, B, C_{in})$
 $C_{out} = F_2(A, B, C_{in})$



$$\begin{aligned} S &= A \oplus B \oplus C_{in} \\ C_{out} &= AB + AC_{in} + BC_{in} \end{aligned}$$

Some Definitions

- **Complement:** variable with a bar over it $\bar{A}, \bar{B}, \bar{C}$
- **Literal:** variable or its complement $A, \bar{A}, B, \bar{B}, C, \bar{C}$
- **Implicant:** product of literals $ABC, \bar{A}\bar{C}, \bar{B}\bar{C}$
- **Minterm:** product that includes all input variables $\bar{A}\bar{B}\bar{C}, \bar{A}\bar{B}C, \bar{A}B\bar{C}, \bar{A}BC, A\bar{B}\bar{C}, A\bar{B}C, AB\bar{C}, ABC$
- **Sum:** the OR of one or more literals
- **Maxterm:** sum that includes all input variables $(\bar{A}+\bar{B}+\bar{C}), (\bar{A}+\bar{B}+C), (\bar{A}+B+\bar{C}), (\bar{A}+B+C), (A+\bar{B}+\bar{C}), (A+\bar{B}+C), (A+B+\bar{C}), (A+B+C)$

Sum-of-Products (SOP) Form

- All equations can be written in SOP form
- Each row has a **minterm**
- A minterm is a product (AND) of literals
- Each minterm is TRUE for that row (and only that row)
- Form function by ORing minterms where the output is TRUE
- Thus, a sum (OR) of products (AND terms)

A	B	Y	minterm	minterm name
0	0	0	$\overline{A} \overline{B}$	m_0
0	1	1	$\overline{A} B$	m_1
1	0	0	$A \overline{B}$	m_2
1	1	1	$A B$	m_3

$$Y = F(A, B) =$$

Sum-of-Products (SOP) Form

- All equations can be written in SOP form
- Each row has a **minterm**
- A minterm is a product (AND) of literals
- Each minterm is TRUE for that row (and only that row)
- Form function by ORing minterms where the output is TRUE
- Thus, a sum (OR) of products (AND terms)

A	B	Y	minterm	minterm name
0	0	0	$\overline{A} \overline{B}$	m_0
0	1	1	$\overline{A} B$	m_1
1	0	0	$A \overline{B}$	m_2
1	1	1	$A B$	m_3

$$Y = F(A, B) =$$

Sum-of-Products (SOP) Form

- All equations can be written in SOP form
- Each row has a **minterm**
- A minterm is a product (AND) of literals
- Each minterm is TRUE for that row (and only that row)
- Form function by ORing minterms where the output is TRUE
- Thus, a sum (OR) of products (AND terms)

A	B	Y	minterm	minterm name
0	0	0	$\overline{A} \overline{B}$	m_0
0	1	1	$\overline{A} B$	m_1
1	0	0	$A \overline{B}$	m_2
1	1	1	$A B$	m_3

$$Y = F(A, B) = \overline{A}B + AB = \Sigma(1, 3)$$

Product-of-Sums (POS) Form

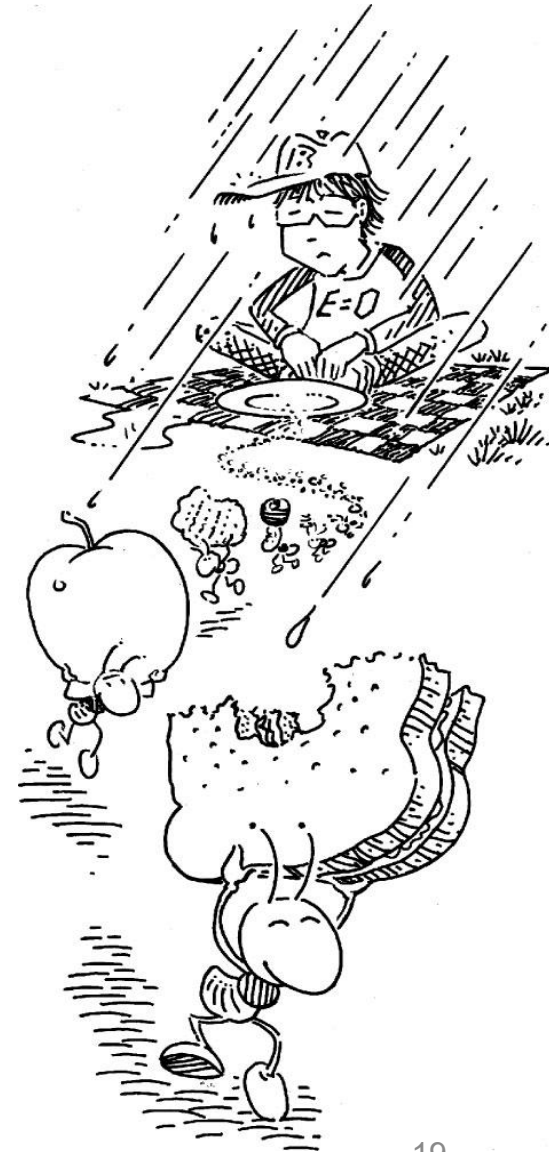
- All Boolean equations can be written in POS form
- Each row has a **maxterm**
- A maxterm is a sum (OR) of literals
- Each maxterm is FALSE for that row (and only that row)
- Form function by ANDing the maxterms for which the output is FALSE
- Thus, a product (AND) of sums (OR terms)

A	B	Y	maxterm	maxterm name
0	0	0	$A + B$	M_0
0	1	1	$A + \overline{B}$	M_1
1	0	0	$\overline{A} + B$	M_2
1	1	1	$\overline{A} + \overline{B}$	M_3

$$Y = F(A, B) = (A + B)(\overline{A} + B) = \Pi(0, 2)$$

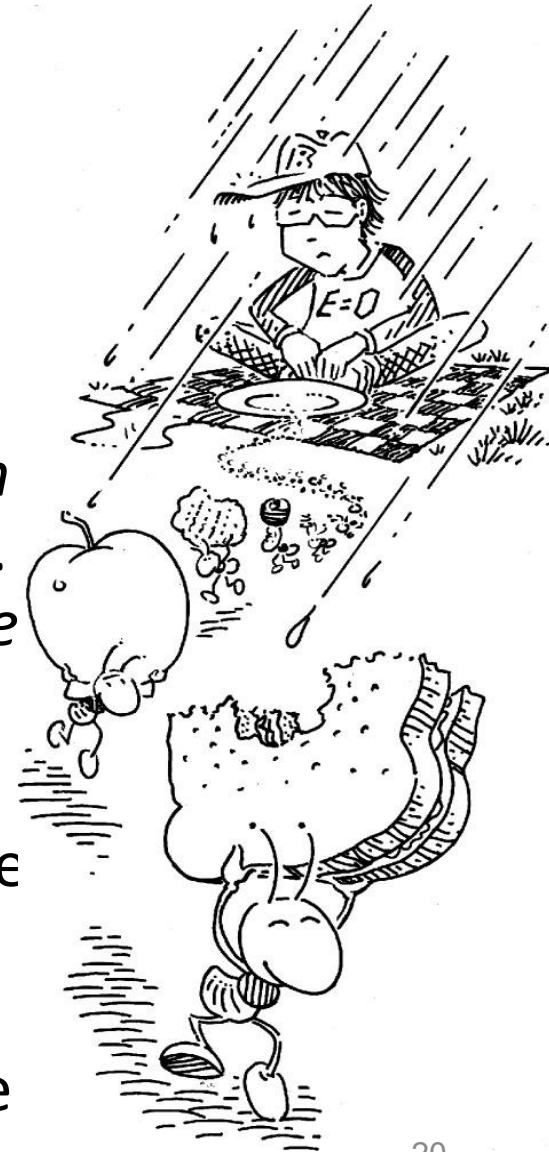
Example 2.2 SUM-OF-PRODUCTS FORM

- Ben Bitdiddle is having a picnic. He won't enjoy it if it rains or if there are ants.
- Design a circuit that will output TRUE *only if Ben enjoys the picnic.*



Example 2.2 SUM-OF-PRODUCTS FORM

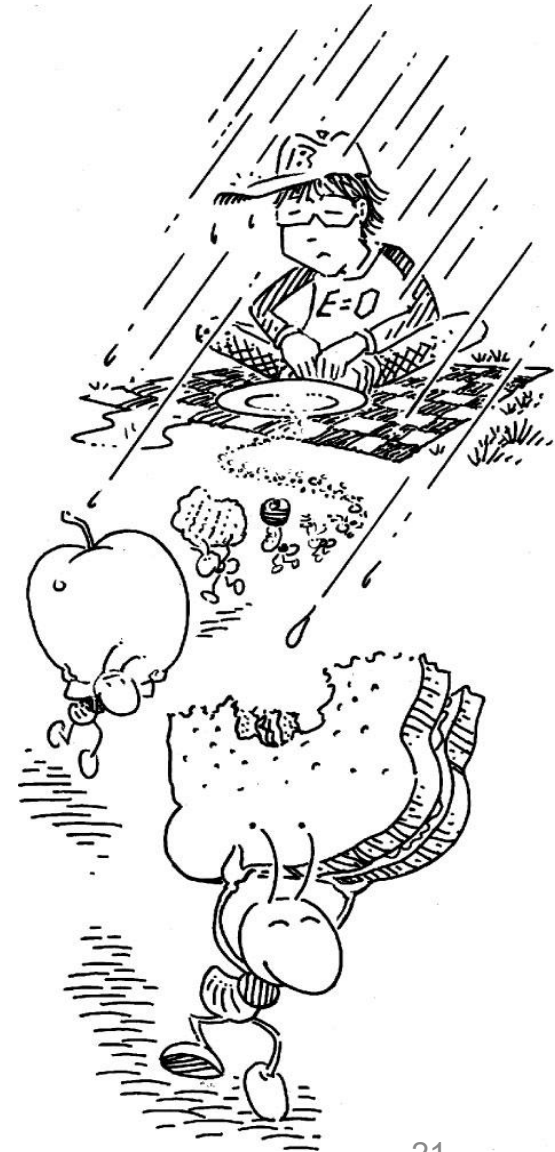
- Ben Bitdiddle is having a picnic. He won't enjoy it if it rains or if there are ants. Design a circuit that will output *TRUE only if Ben enjoys the picnic.*
- **Solution:** ① define the inputs and outputs. The inputs are *A* and *R*, which indicate if there are ants and if it rains. The output is *E*, Ben's enjoyment of the picnic.
- ② *A* is *TRUE* when there are ants and *FALSE* when there are no ants. Likewise *R* is *TRUE* when it rains and *FALSE* when the sun smiles on Ben. *E* is *TRUE* if Ben enjoys the picnic and *FALSE* if he suffers.



Example 2.2 SUM-OF-PRODUCTS FORM

- Ben Bitdiddle is having a picnic. He won't enjoy it if it rains or if there are ants. Design a circuit that will output **TRUE** *only if Ben enjoys the picnic*.
- Solution:** ③ Figure shows the truth table for Ben's picnic experience.

<i>A</i>	<i>R</i>	<i>E</i>
0	0	1
0	1	0
1	0	0
1	1	0

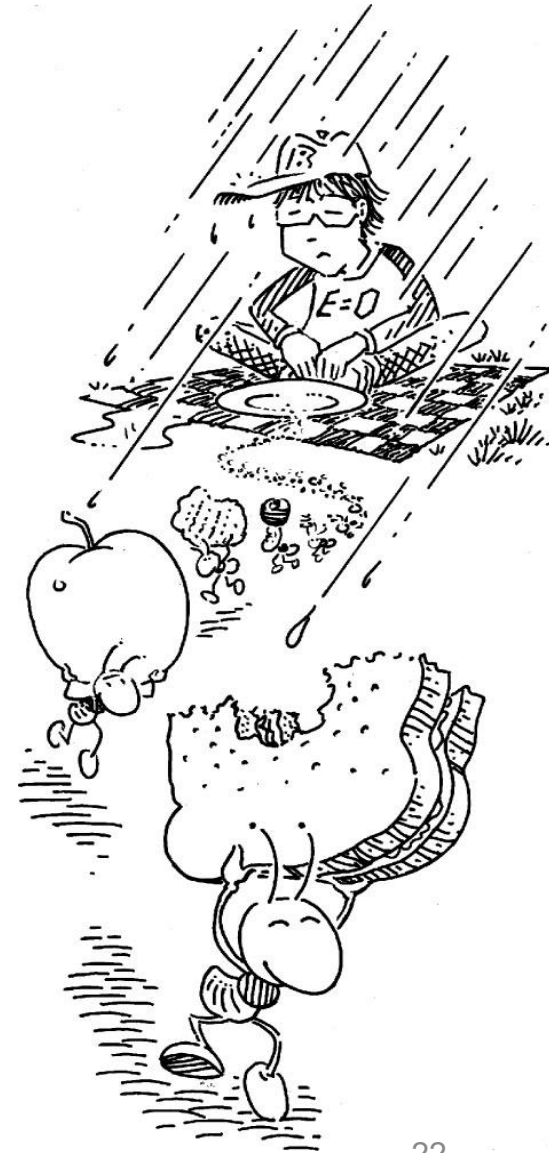


Example 2.2 SUM-OF-PRODUCTS FORM

- Ben Bitdiddle is having a picnic. He won't enjoy it if it rains or if there are ants. Design a circuit that will output TRUE *only if Ben enjoys the picnic.*
- Solution:** ④ Using sum-of-products form, we write the equation as:

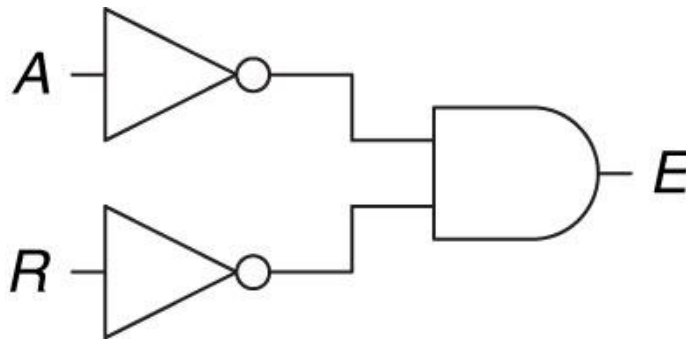
<i>A</i>	<i>R</i>	<i>E</i>
0	0	1
0	1	0
1	0	0
1	1	0

$$E = \overline{A} \overline{R}$$

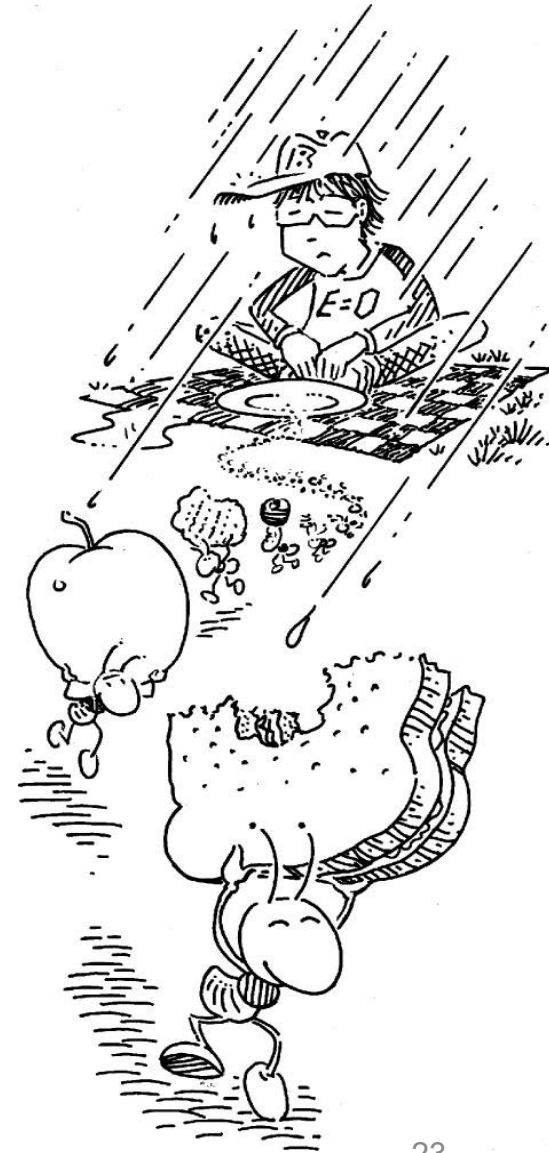


Example 2.2 SUM-OF-PRODUCTS FORM

- Ben Bitdiddle is having a picnic. He won't enjoy it if it rains or if there are ants. Design a circuit that will output **TRUE** *only if Ben enjoys the picnic.*
- **Solution:** $E = \overline{A} \overline{R}$
- ⑤ We can build the equation using two inverters and a two-input AND gate, shown in Figure



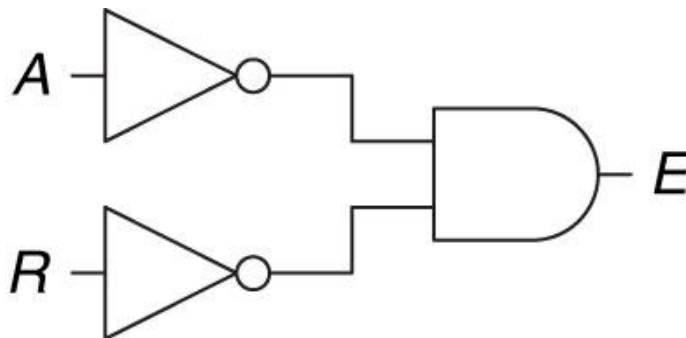
(a)



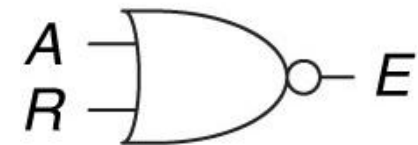
Example 2.2 SUM-OF-PRODUCTS FORM

- Ben Bitdiddle is having a picnic. He won't enjoy it if it rains or if there are ants. Design a circuit that will output TRUE *only if Ben enjoys the picnic.*
- Solution:** $E = \overline{A} \overline{R}$
- ⑤ We can build the equation using two inverters and a two-input AND gate, shown in Figure

<i>A</i>	<i>R</i>	<i>E</i>
0	0	1
0	1	0
1	0	0
1	1	0



(a)



(b)

Boolean Equations Example

- You are going to the cafeteria for lunch
 - You won't eat lunch ($\bar{E}$)
 - If it's not open ($\bar{O}$) or
 - If they only serve corndogs (C)
- Write a truth table for determine whether if you will eat lunch (E).

O	C	E
0	0	
0	1	
1	0	
1	1	

Boolean Equations Example

- You are going to the cafeteria for lunch
You won't eat lunch ($\bar{E}$)
 - If it's not open ($\bar{O}$) or
 - If they only serve corndogs (C)
- Write a truth table for determining if you will eat lunch (E).

O	C	E
0	0	0
0	1	0
1	0	1
1	1	0

SOP & POS Form

- SOP – sum-of-products

O	C	E	minterm
0	0		$\overline{O} \overline{C}$
0	1		$\overline{O} C$
1	0		$O \overline{C}$
1	1		$O C$

- POS – product-of-sums

O	C	Y	maxterm
0	0		$O + C$
0	1		$O + \overline{C}$
1	0		$\overline{O} + C$
1	1		$\overline{O} + \overline{C}$

SOP & POS Form

- SOP – sum-of-products

O	C	E	minterm
0	0	0	$\overline{O} \overline{C}$
0	1	0	$\overline{O} C$
1	0	1	$O \overline{C}$
1	1	0	$O C$

$$Y = \overline{O} \overline{C}$$

$$= \Sigma(2)$$

- POS – product-of-sums

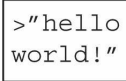
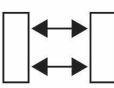
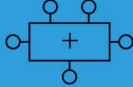
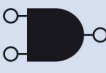
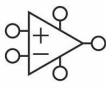
O	C	E	maxterm
0	0	0	$O + C$
0	1	0	$O + \overline{C}$
1	0	1	$\overline{O} + C$
1	1	0	$\overline{O} + \overline{C}$

$$Y = (O + C)(O + \overline{C})(\overline{O} + \overline{C})$$

$$= \Pi(0, 1, 3)$$

Chapter 2 :: Topics

- Introduction
- Boolean Equations
- **Boolean Algebra**
- From Logic to Gates
- Combinational Logic
- X's and Z's
- Karnaugh Maps
- Combinational Building Blocks
- Timing

Application Software	
Operating Systems	
Architecture	
Micro-architecture	
Logic	
Digital Circuits	
Analog Circuits	
Devices	
Physics	

Boolean Algebra

- Axioms and theorems to **simplify** Boolean equations
- Like regular algebra, but simpler: variables have only two values (1 or 0)
- **Duality** in axioms and theorems:
 - ANDs and ORs, 0's and 1's interchanged

Boolean Axioms

Axiom		Dual		Name
A1	$B = 0 \text{ if } B \neq 1$	A1'	$B = 1 \text{ if } B \neq 0$	Binary field
A2	$\overline{0} = 1$	A2'	$\overline{1} = 0$	NOT
A3	$0 \bullet 0 = 0$	A3'	$1 + 1 = 1$	AND/OR
A4	$1 \bullet 1 = 1$	A4'	$0 + 0 = 0$	AND/OR
A5	$0 \bullet 1 = 1 \bullet 0 = 0$	A5'	$1 + 0 = 0 + 1 = 1$	AND/OR

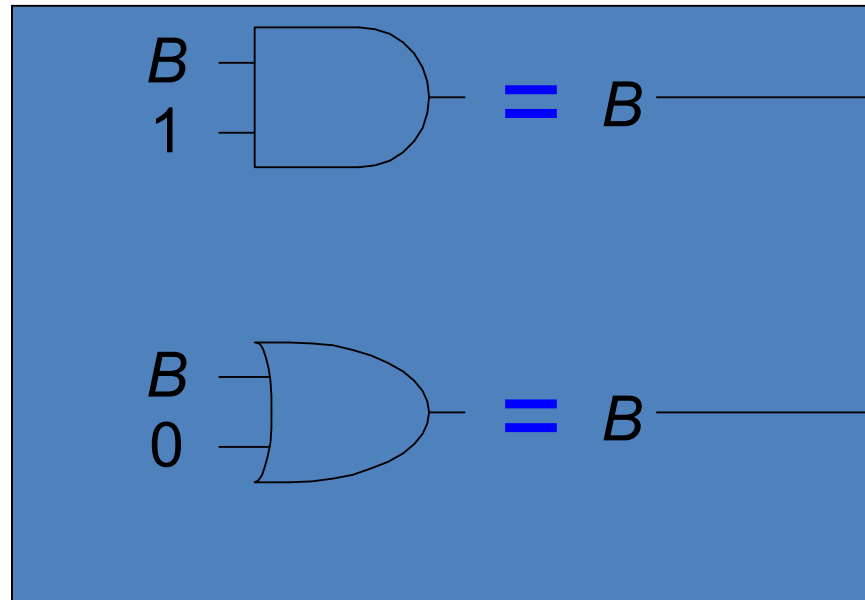
Theorem		Dual		Name
T1	$B \bullet 1 = B$	T1'	$B + 0 = B$	Identity
T2	$B \bullet 0 = 0$	T2'	$B + 1 = 1$	Null Element
T3	$B \bullet B = B$	T3'	$B + B = B$	Idempotency
T4		$\overline{\overline{B}} = B$		Involution
T5	$B \bullet \overline{B} = 0$	T5'	$B + \overline{B} = 1$	Complements

T1: Identity Theorem(恒等, 同一)

- $B \cdot 1 = B$
- $B + 0 = B$

T1: Identity Theorem

- $B \cdot 1 = B$
- $B + 0 = B$

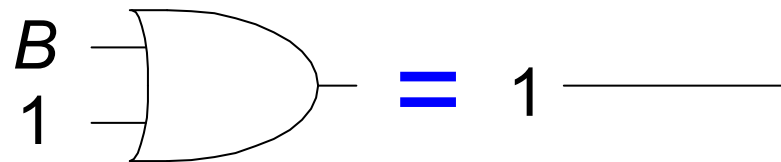
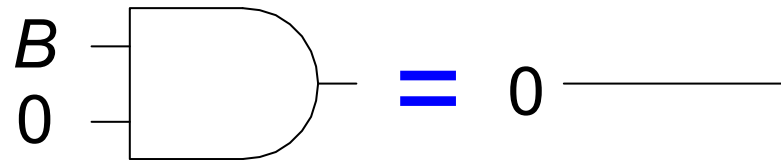


T2: Null Element Theorem (零元)

- $B \cdot 0 = 0$
- $B + 1 = 1$

T2: Null Element Theorem

- $B \cdot 0 = 0$
- $B + 1 = 1$

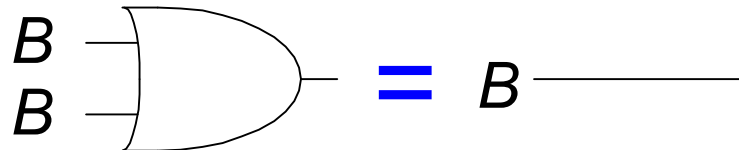
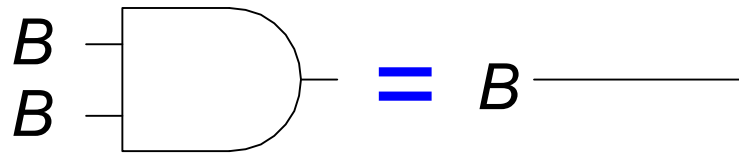


T3: Idempotency Theorem (幂等)

- $B \cdot B = B$
- $B + B = B$

T3: Idempotency Theorem

- $B \cdot B = B$
- $B + B = B$

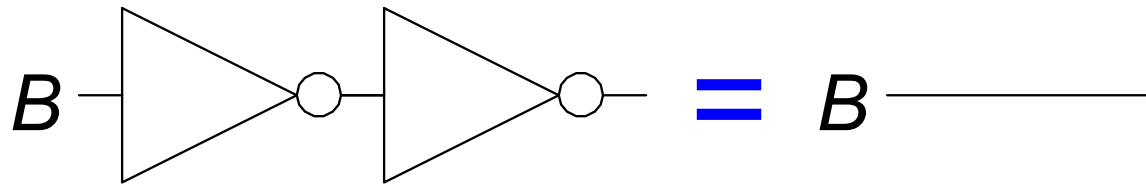


T4: involution Theorem (复合, 卷绕)

- $\overline{\overline{B}} = B$

T4: involution Theorem

- $\overline{\overline{B}} = B$

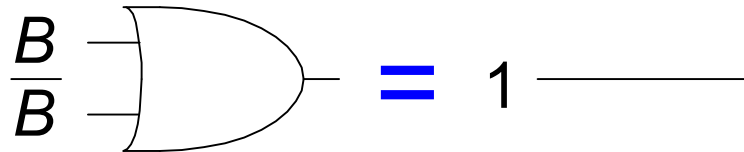
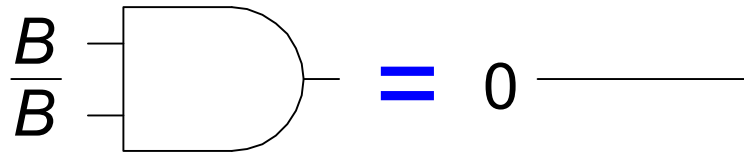


T5: Complement Theorem

- $B \cdot \bar{B} = 0$
- $B + \bar{B} = 1$

T5: Complement Theorem

- $B \cdot \bar{B} = 0$
- $B + \bar{B} = 1$



Boolean Theorems Summary

	Theorem		Dual	Name
T1	$B \bullet 1 = B$	T1'	$B + 0 = B$	Identity
T2	$B \bullet 0 = 0$	T2'	$B + 1 = 1$	Null Element
T3	$B \bullet B = B$	T3'	$B + B = B$	Idempotency
T4		$\overline{\overline{B}} = B$		Involution
T5	$B \bullet \overline{B} = 0$	T5'	$B + \overline{B} = 1$	Complements

Boolean Theorems of Several Vars

Theorem		Dual		Name
T6	$B \bullet C = C \bullet B$	T6'	$B + C = C + B$	Commutativity
T7	$(B \bullet C) \bullet D = B \bullet (C \bullet D)$	T7'	$(B + C) + D = B + (C + D)$	Associativity
T8	$(B \bullet C) + B \bullet D = B \bullet (C + D)$	T8'	$(B + C) \bullet (B + D) = B + (C \bullet D)$	Distributivity
T9	$B \bullet (B + C) = B$	T9'	$B + (B \bullet C) = B$	Covering
T10	$(B \bullet C) + (B \bullet \overline{C}) = B$	T10'	$(B + C) \bullet (B + \overline{C}) = B$	Combining
T11	$(B \bullet C) + (\overline{B} \bullet D) + (C \bullet D)$ $= B \bullet C + \overline{B} \bullet D$	T11'	$(B + C) \bullet (\overline{B} + D) \bullet (C + D)$ $= (B + C) \bullet (\overline{B} + D)$	Consensus
T12	$\overline{B_0 \bullet B_1 \bullet B_2 \dots}$ $= (\overline{B_0} + \overline{B_1} + \overline{B_2} \dots)$	T12'	$\overline{B_0 + B_1 + B_2 \dots}$ $= (\overline{B_0} \bullet \overline{B_1} \bullet \overline{B_2})$	De Morgan's Theorem

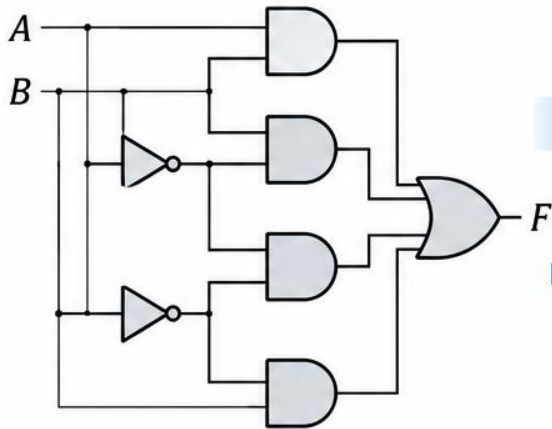
Boolean Algebra

are powerful utilities for simplify boolean equation.

*Simpler Logic
A Smarter World*

A Complex Boolean Expression

$$F = A \cdot B + A \cdot \bar{B} + \bar{A} \cdot B + \bar{A} \cdot \bar{B}$$



→
Simplify
with
Boolean Algebra

A Simple Equivalent Expression

$$F = 1$$

1 ————— F

Complicated form

(more gates, more cost, harder to understand)

Simplified form

(minimal implementation)

Why simplify Boolean equation?



Fewer Gates

Less hardware, lower cost.



Higher Efficiency

Lower power consumption and faster operation.



Easier to Design

Simpler circuits are easier to implement and verify.



More Reliable

Fewer components, lower error probability.

Simple Logic. Big Impact.
From Boolean Algebra to Real-World Systems.



Simplifying Boolean Equations

Example 1:

- $Y = AB + \overline{A}B$

Simplifying Boolean Equations

Example 1:

- $Y = AB + \overline{A}B$

$$= B(A + \overline{A}) \quad \text{T8}$$

$$= B(1) \quad \text{T5'}$$

$$= B \quad \text{T1}$$

Simplifying Boolean Equations

Example 2:

- $Y = A(AB + ABC)$

Simplifying Boolean Equations

Example 2:

- $Y = A(AB + ABC)$

$$= A(AB(1 + C)) \quad \text{T8}$$

$$= A(AB(1)) \quad \text{T2'}$$

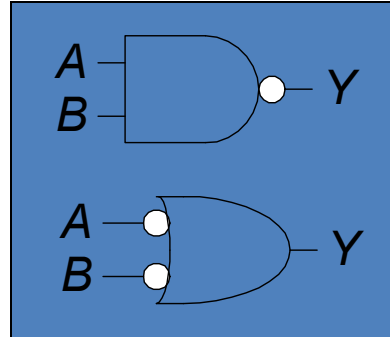
$$= A(AB) \quad \text{T1}$$

$$= (AA)B \quad \text{T7}$$

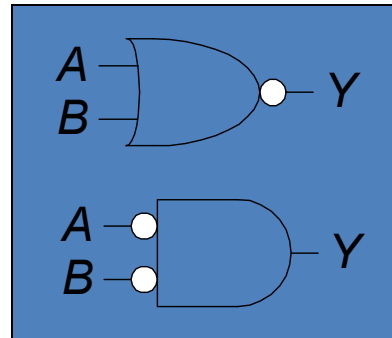
$$= AB \quad \text{T3}$$

DeMorgan's Theorem

- $Y = \overline{AB} = \overline{A} + \overline{B}$



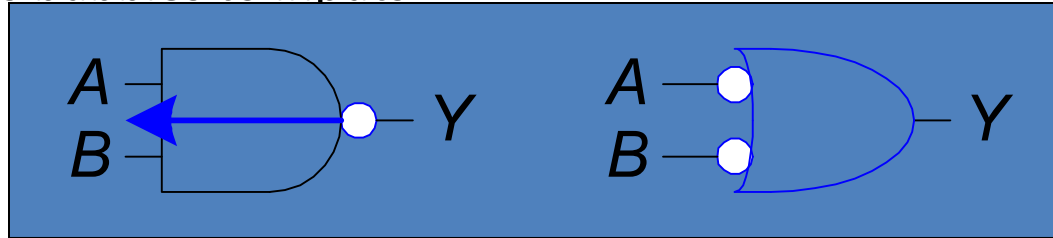
- $Y = \overline{A + B} = \overline{A} \cdot \overline{B}$



Bubble Pushing

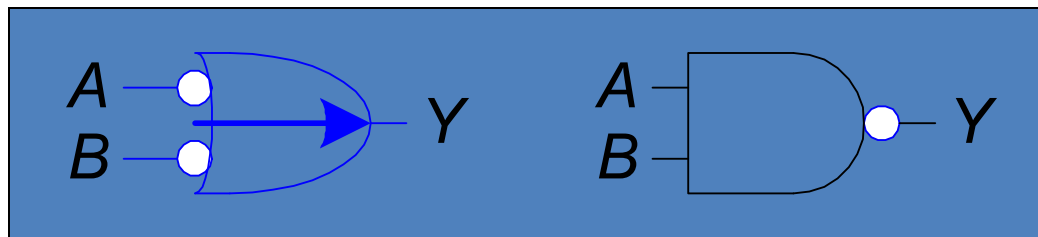
- **Backward:**

- Body changes
- Adds bubbles to inputs



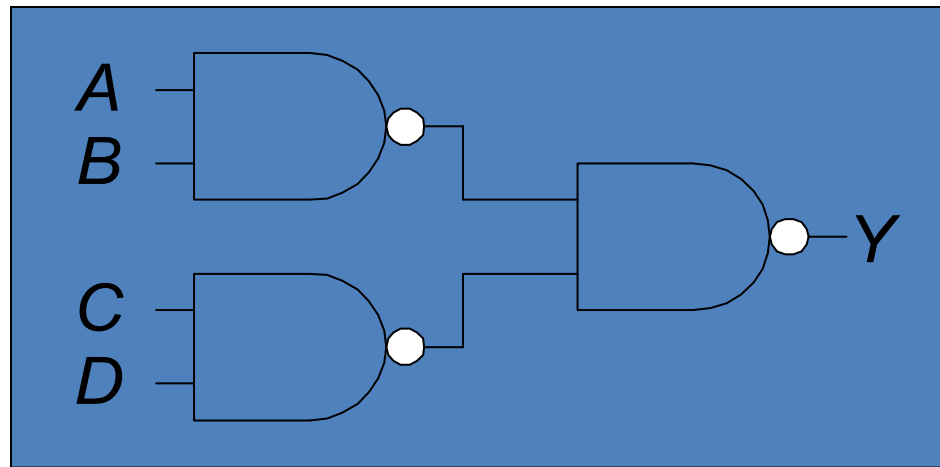
- **Forward:**

- Body changes
- Adds bubble to output



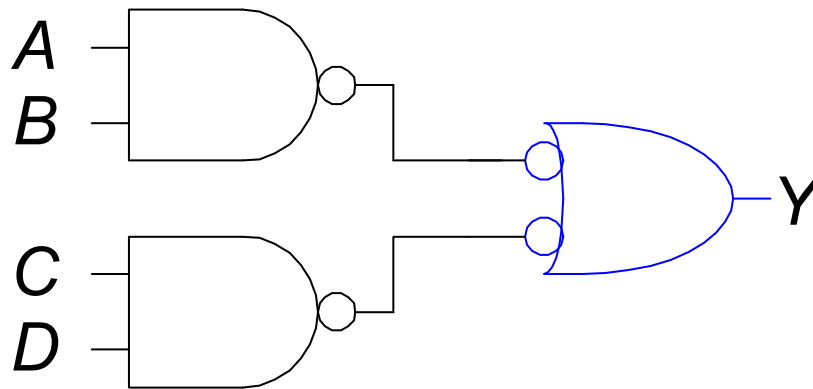
Bubble Pushing

- What is the Boolean expression for this circuit?



Bubble Pushing

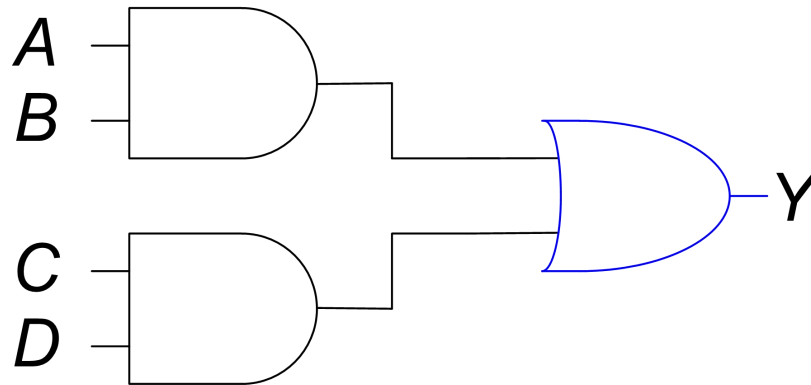
- What is the Boolean expression for this circuit?



$$Y = AB + CD$$

Bubble Pushing

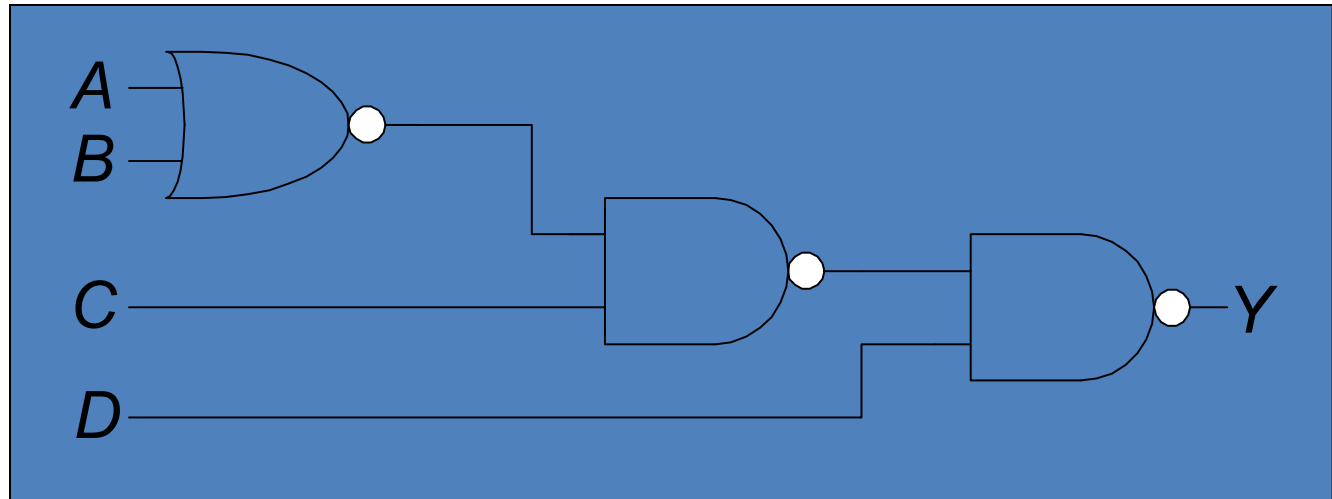
- What is the Boolean expression for this circuit?



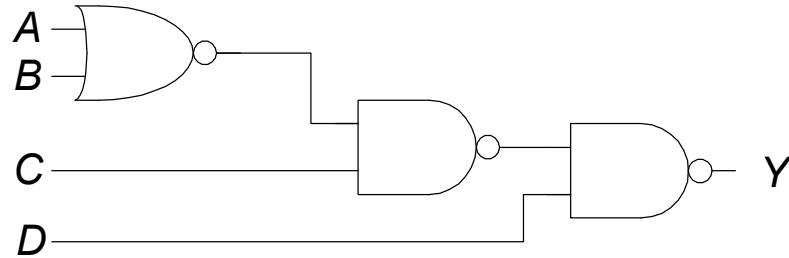
$$Y = AB + CD$$

Bubble Pushing Rules

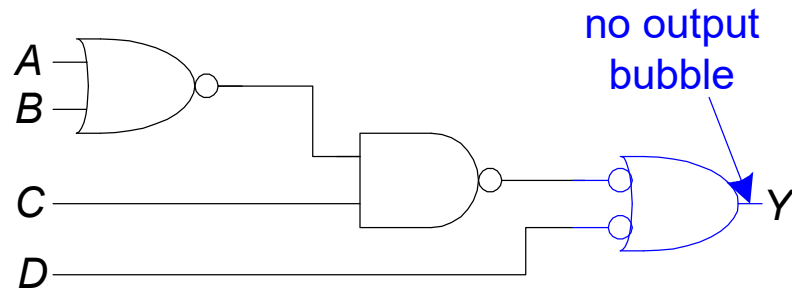
- Begin at output, then work toward inputs
- Push bubbles on final output back
- Draw gates in a form so bubbles cancel



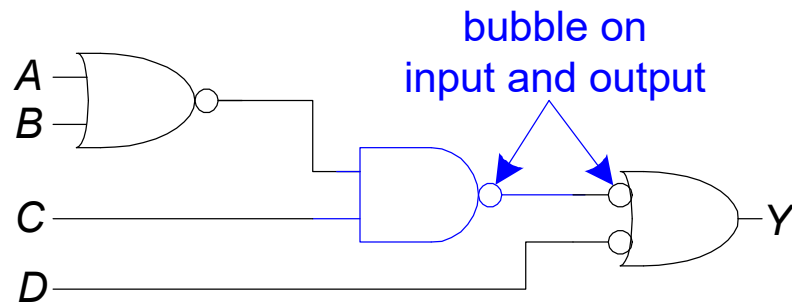
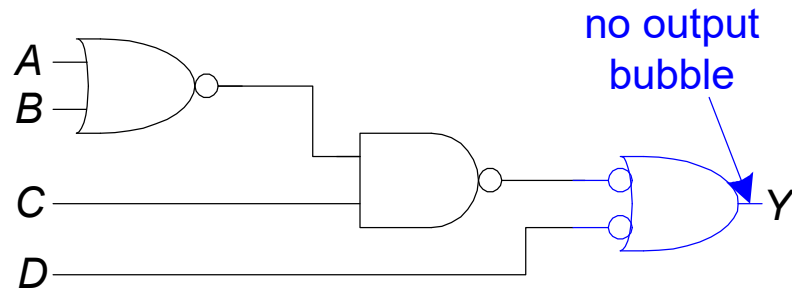
Bubble Pushing Example



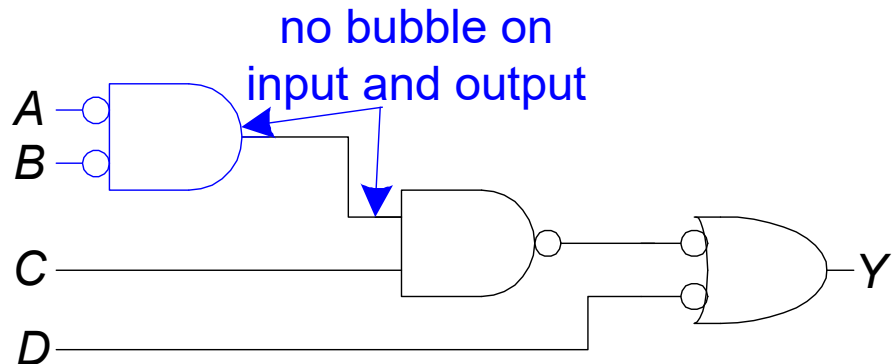
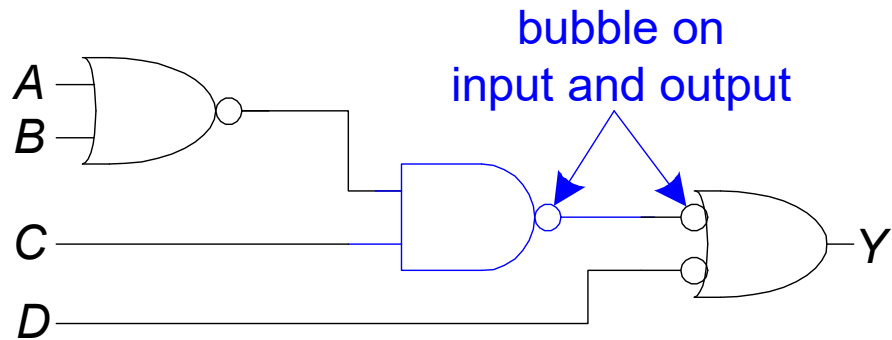
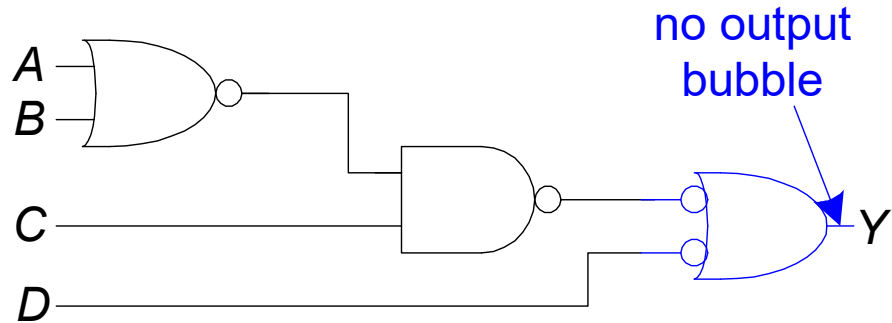
Bubble Pushing Example



Bubble Pushing Example



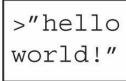
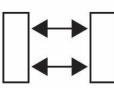
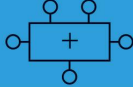
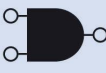
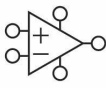
Bubble Pushing Example



$$Y = \overline{A}\overline{B}C + \overline{D}$$

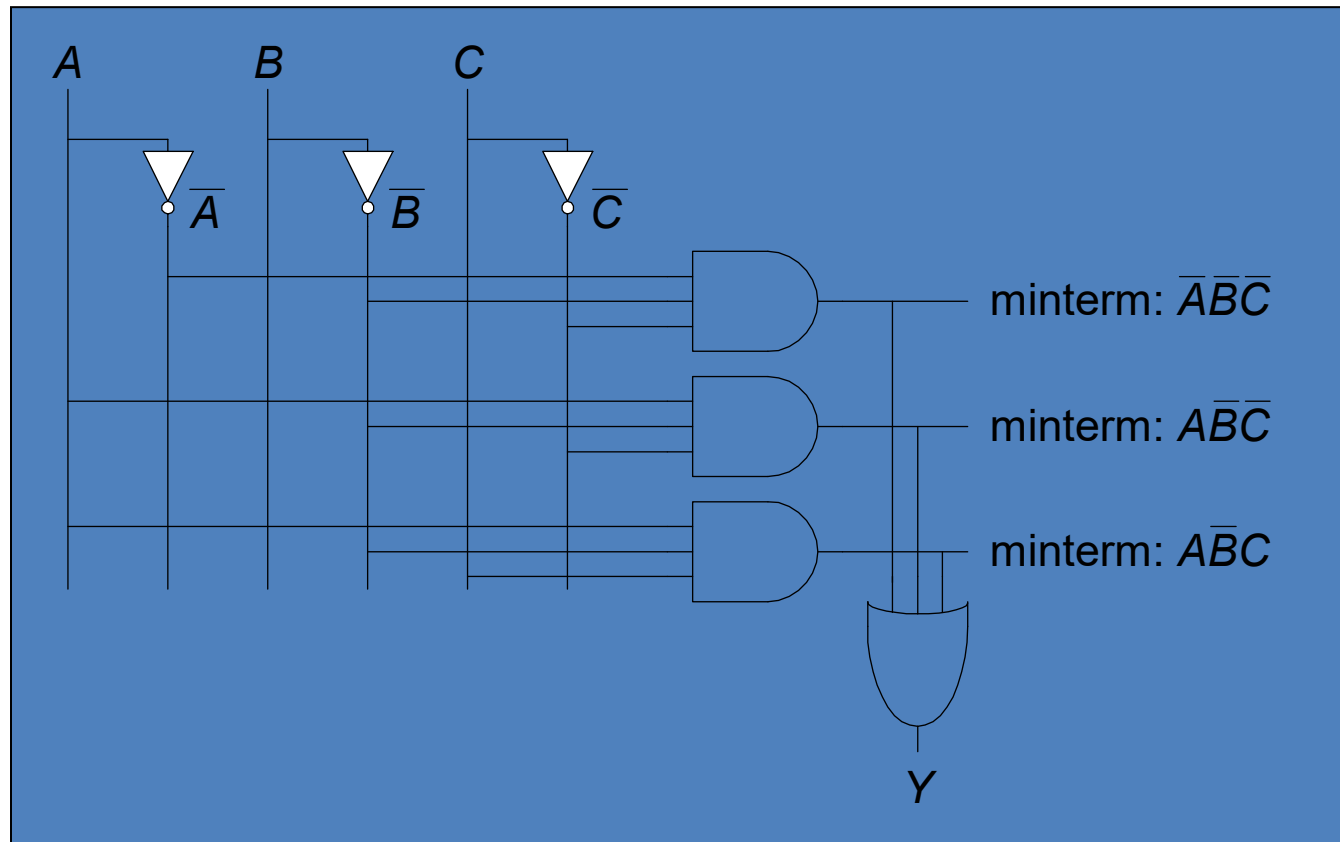
Chapter 2 :: Topics

- Introduction
- Boolean Equations
- Boolean Algebra
- **From Logic to Gates**
- Combinational Logic
- X's and Z's
- Karnaugh Maps
- Combinational Building Blocks
- Timing

Application Software	
Operating Systems	
Architecture	
Micro-architecture	
Logic	
Digital Circuits	
Analog Circuits	
Devices	
Physics	

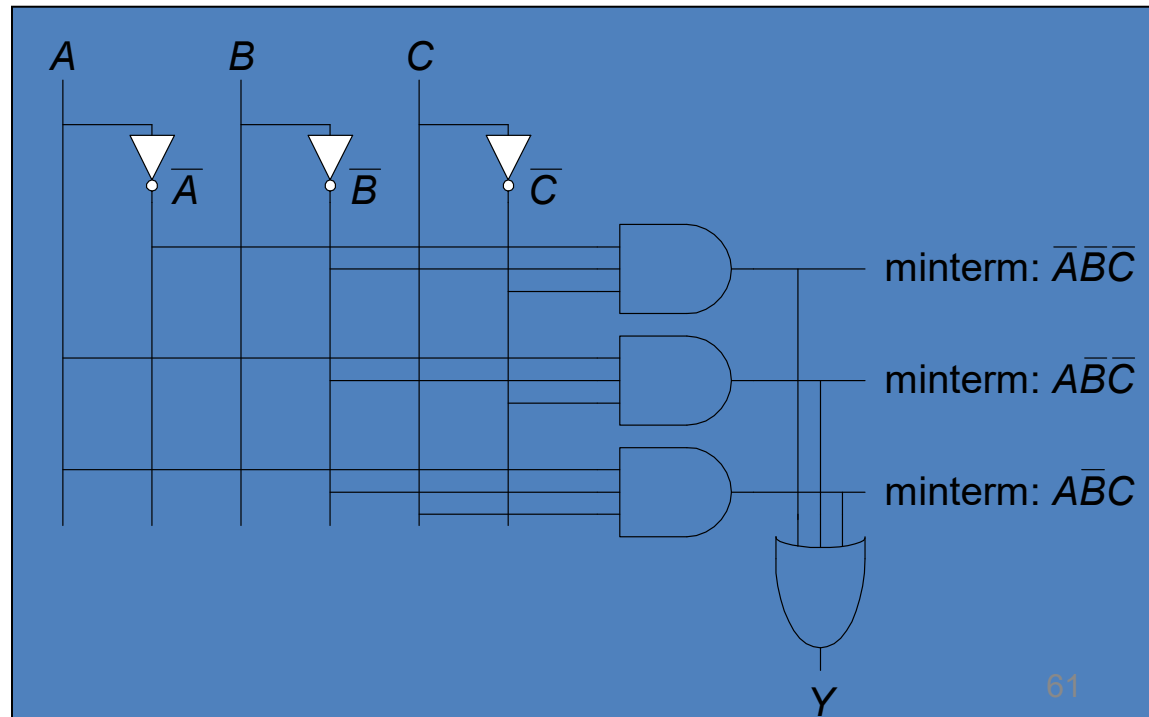
From Logic to Gates

- Two-level logic: ANDs followed by ORs
- Example: $Y = \bar{A}\bar{B}\bar{C} + A\bar{B}\bar{C} + A\bar{B}C$



Circuit Schematics Rules

- Inputs on the left (or top)
- Outputs on right (or bottom)
- Gates flow from left to right
- Straight wires are best



Morris Chang

1931 –

A global semiconductor pioneer
who built a brighter future.

*Semiconductors
Empower a Better World*

1931	1949	1950	1954	1958	1964	1972	1985	1987	2018
Born in Ningbo, China	Harvard University	MIT (mech. eng.)	Bachelor's degree (MIT)	Joined Texas Instruments	Ph.D. (EE), Stanford University	Vice President of TI	Back to Taiwan	Founded TSMC	Retired
									

*"Build things
that last."
Morris Chang*



TSMC

A new foundry model.
A stronger, more connected world.

*From Taiwan
to the World*

Innovate
Enable
Empower

Semiconductor technology
connects people, industries and a better future.



TSMC in the Semiconductor Industry

A pure-play foundry connecting global innovation to a smarter world.

*Together
We Enable
a Better Future*

Upstream

Design & Enable

EDA Tools

synopsys[®] cadence[®]
SIEMENS

IP Cores

arm[®] synopsys[®]
Rambus

Materials

ShinEtsu
SUMITOMO CHEMICAL
FUJIFILM

Equipment

ASML
APPLIED MATERIALS
TEL[®]
TOKYO ELECTRON

Fabless Chip Design

Design the chips



Design advanced chips
(e.g., CPUs, GPUs, SoCs)
but do not manufacture
them.

TSMC

Manufacture the chips



The world's leading
pure-play foundry



Turn designs into high-quality,
advanced chips through
world-class manufacturing.

Downstream

Assembly, Packaging & Testing



Package, test and deliver
chips for use in electronic
products.

System & Product Companies

Integrate chips into end products



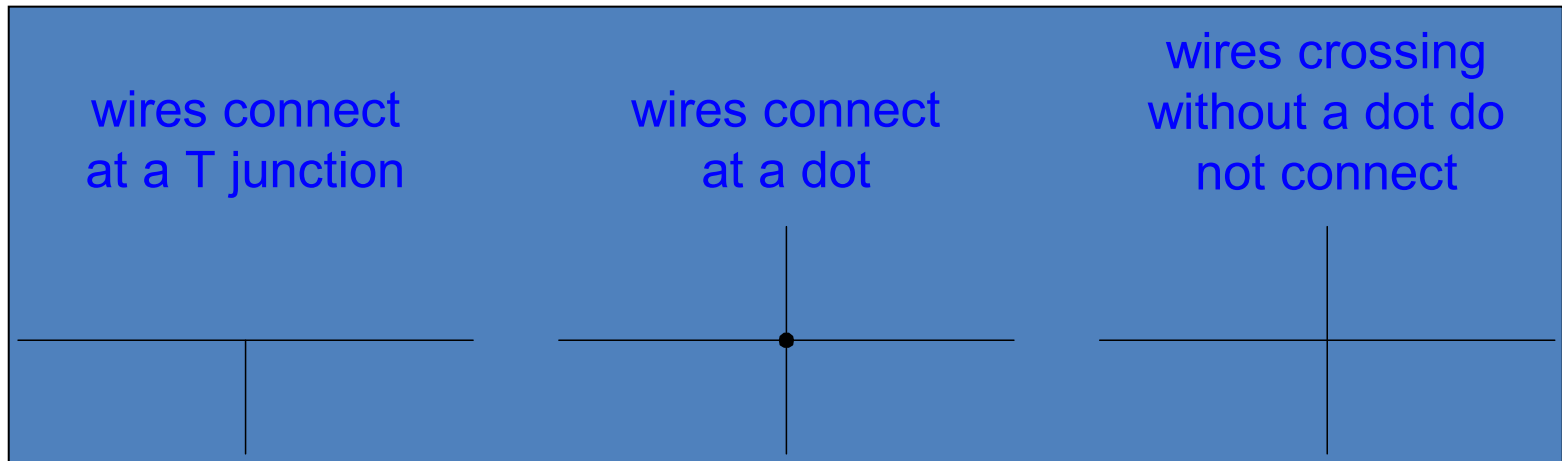
Use chips (and other components)
to build complete systems and
consumer products, such as PCs,
laptops, servers, smartphones, etc.

TSMC does not compete with its customers. It enables them.

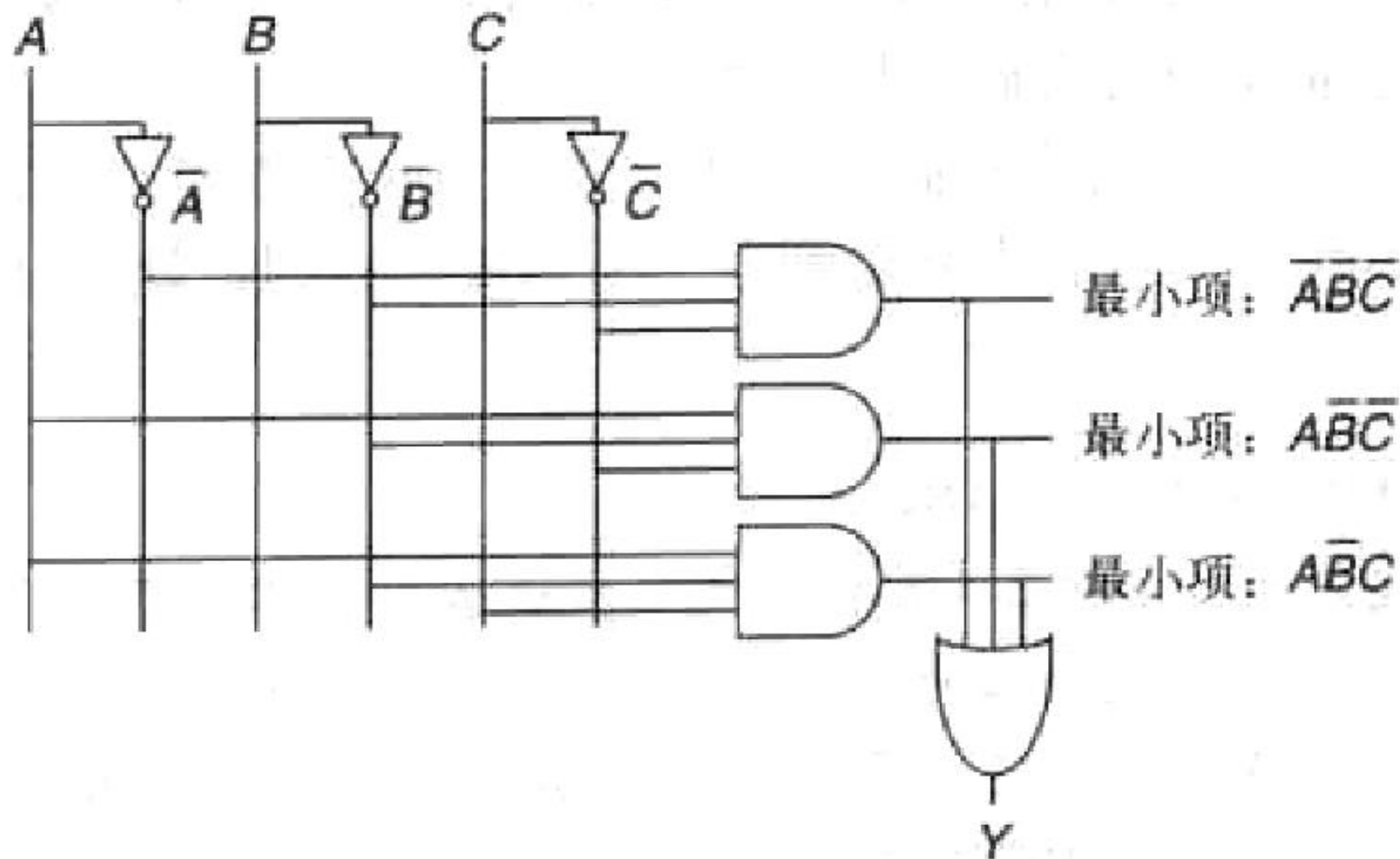
From chips to countless possibilities.

Circuit Schematic Rules (cont.)

- Wires always connect at a T junction
- A dot where wires cross indicates a connection between the wires
- Wires crossing *without* a dot make no connection



$$\bar{A}\bar{B}\bar{C} + A\bar{B}\bar{C} + A\bar{B}C$$



simplification

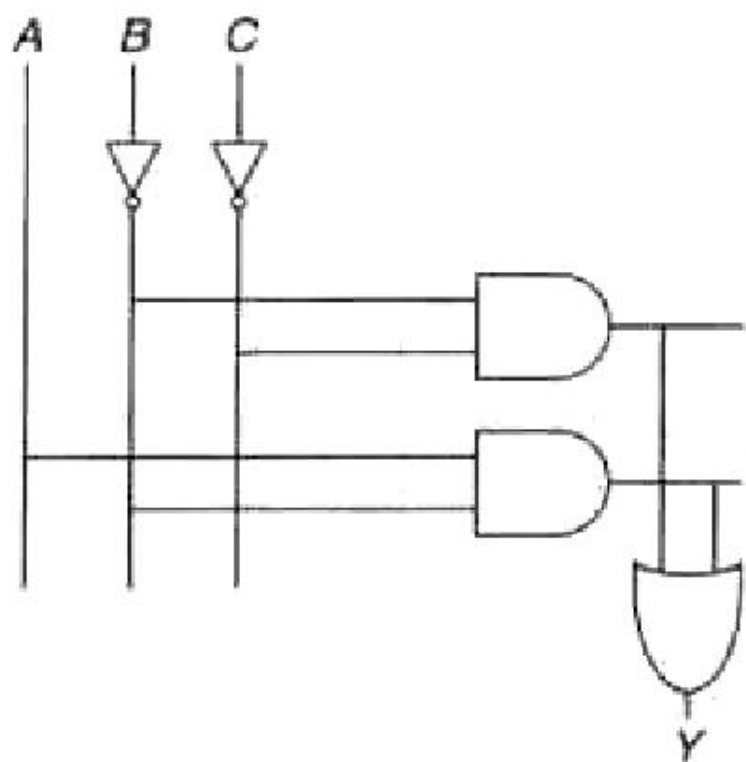
	$\bar{A}\bar{B}\bar{C} + A\bar{B}\bar{C} + A\bar{B}C$	
1	$\bar{B}\bar{C} (\bar{A} + A) + A\bar{B}C$	T8: 分配律
2	$\bar{B}\bar{C} (1) + A\bar{B}C$	T5: 互补定理
3	$\bar{B}\bar{C} + A\bar{B}C$	T1: 同一性定理

simplification

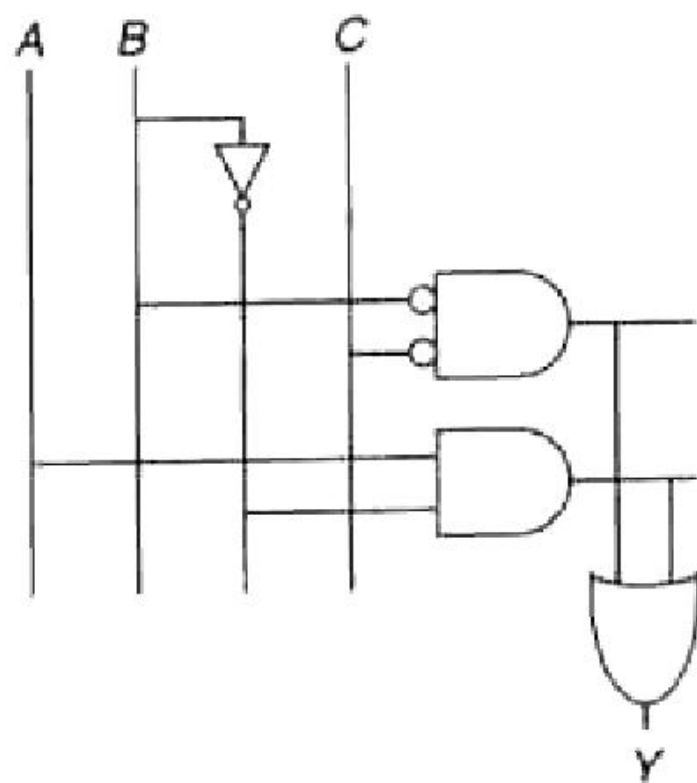
步骤	等 式	定 理
	$\bar{A}\bar{B}\bar{C} + A\bar{B}\bar{C} + \bar{A}BC$	
1	$\bar{A}\bar{B}\bar{C} + A\bar{B}\bar{C} + \bar{A}B\bar{C} + A\bar{B}C$	T3: 重叠定理
2	$\bar{B}\bar{C}(\bar{A} + A) + A\bar{B}(\bar{C} + C)$	T8: 分配律
3	$\bar{B}\bar{C}(1) + A\bar{B}(1)$	T5: 互补定理
4	$\bar{B}\bar{C} + A\bar{B}$	T1: 同一性定理



why we simplify it ?



$$Y = \bar{B}\bar{C} + A\bar{B}$$



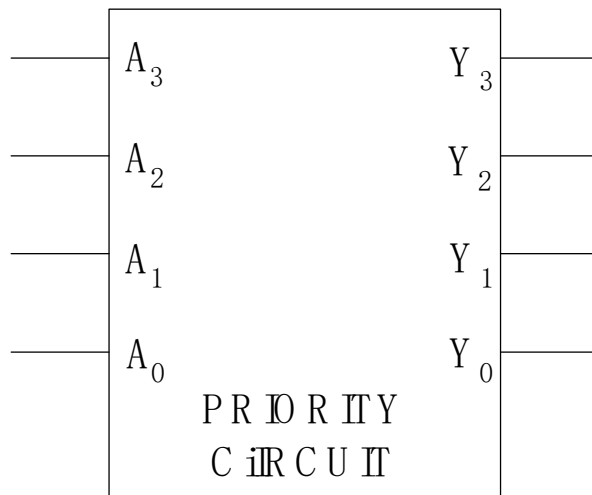
Example 2.7 Multiple-Output Circuits

- **Example: Priority Circuit**
- The dean, the department chair, the teaching assistant, and the dorm social chair, each one use the auditorium from time to time.
- Unfortunately, they occasionally conflict, leading to disasters such as the one that occurred when the dean's fundraising meeting with crusty trustees happened at the same time as the dorm's party.
- Alyssa P. Hacker has been called in to design a room **reservation** system.

Example 2.7 Multiple-Output Circuits

•Example: Priority Circuit

Output asserted
corresponding to
most significant
TRUE input

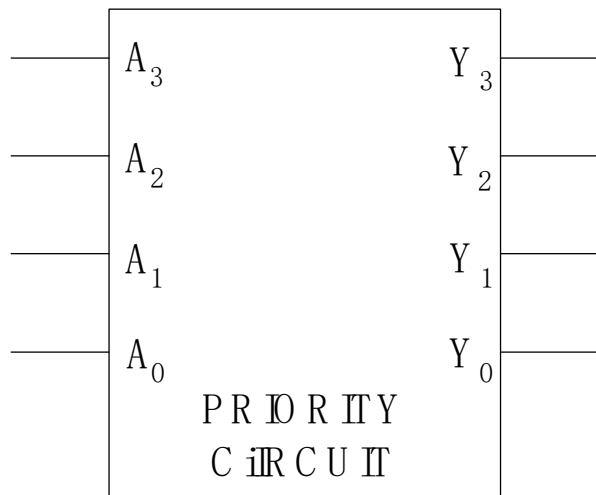


A_3	A_2	A_1	A_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0				
0	0	0	1				
0	0	1	0				
0	0	1	1				
0	1	0	0				
0	1	0	1				
0	1	1	0				
0	1	1	1				
1	0	0	0				
1	0	0	1				
1	0	1	0				
1	0	1	1				
1	1	0	0				
1	1	0	1				
1	1	1	0				
1	1	1	1				
1	1	1	1				

Example 2.7 Multiple-Output Circuits

•Example: Priority Circuit

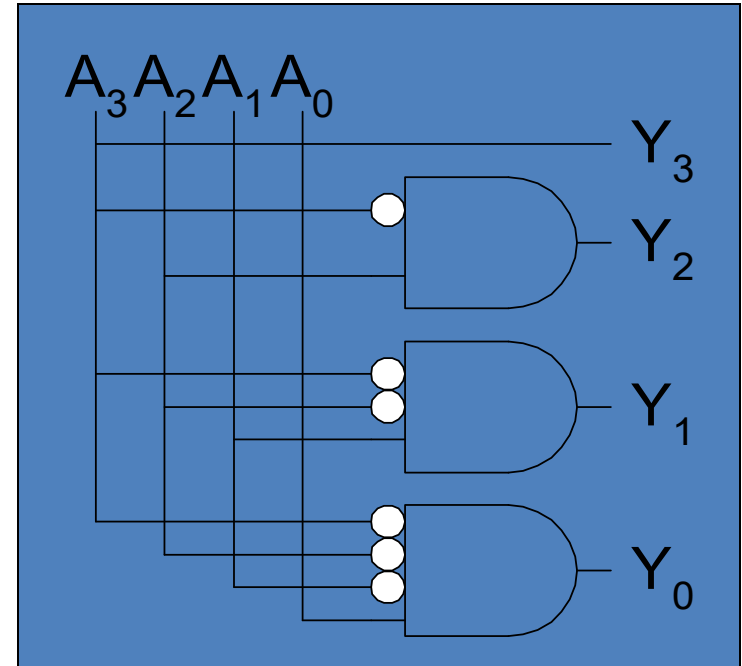
Output asserted
corresponding to
most significant
TRUE input



A_3	A_2	A_1	A_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	0	0	0	1	0
0	0	1	1	0	0	1	0
0	1	0	0	0	1	0	0
0	1	0	1	0	1	0	0
0	1	1	0	0	1	0	0
0	1	1	1	0	1	0	0
1	0	0	0	1	0	0	0
1	0	0	1	1	0	0	0
1	0	1	0	1	0	0	0
1	0	1	1	1	0	0	0
1	1	0	0	1	0	0	0
1	1	0	1	1	0	0	0
1	1	1	0	1	0	0	0
1	1	1	1	1	0	0	0

Priority Circuit Hardware

A_3	A_2	A_1	A_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	0	0	0	1	0
0	0	1	1	0	0	1	0
0	1	0	0	0	1	0	0
0	1	0	1	0	1	0	0
0	1	1	0	0	1	0	0
0	1	1	1	0	1	0	0
1	0	0	0	1	0	0	0
1	0	0	1	1	0	0	0
1	0	1	0	1	0	0	0
1	0	1	1	1	0	0	0
1	1	0	0	1	0	0	0
1	1	0	1	1	0	0	0
1	1	1	0	1	0	0	0
1	1	1	1	1	0	0	0



Don't Cares

A_3	A_2	A_1	A_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	0	0	0	1	0
0	0	1	1	0	0	1	0
0	1	0	0	0	1	0	0
0	1	0	1	0	1	0	0
0	1	1	0	0	1	0	0
0	1	1	1	0	1	0	0
1	0	0	0	1	0	0	0
1	0	0	1	1	0	0	0
1	0	1	0	1	0	0	0
1	0	1	1	1	0	0	0
1	1	0	0	1	0	0	0
1	1	0	1	1	0	0	0
1	1	1	0	1	0	0	0
1	1	1	1	1	0	0	0

A_3	A_2	A_1	A_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	X	0	0	1	0
0	1	X	X	0	1	0	0
1	X	X	X	1	0	0	0

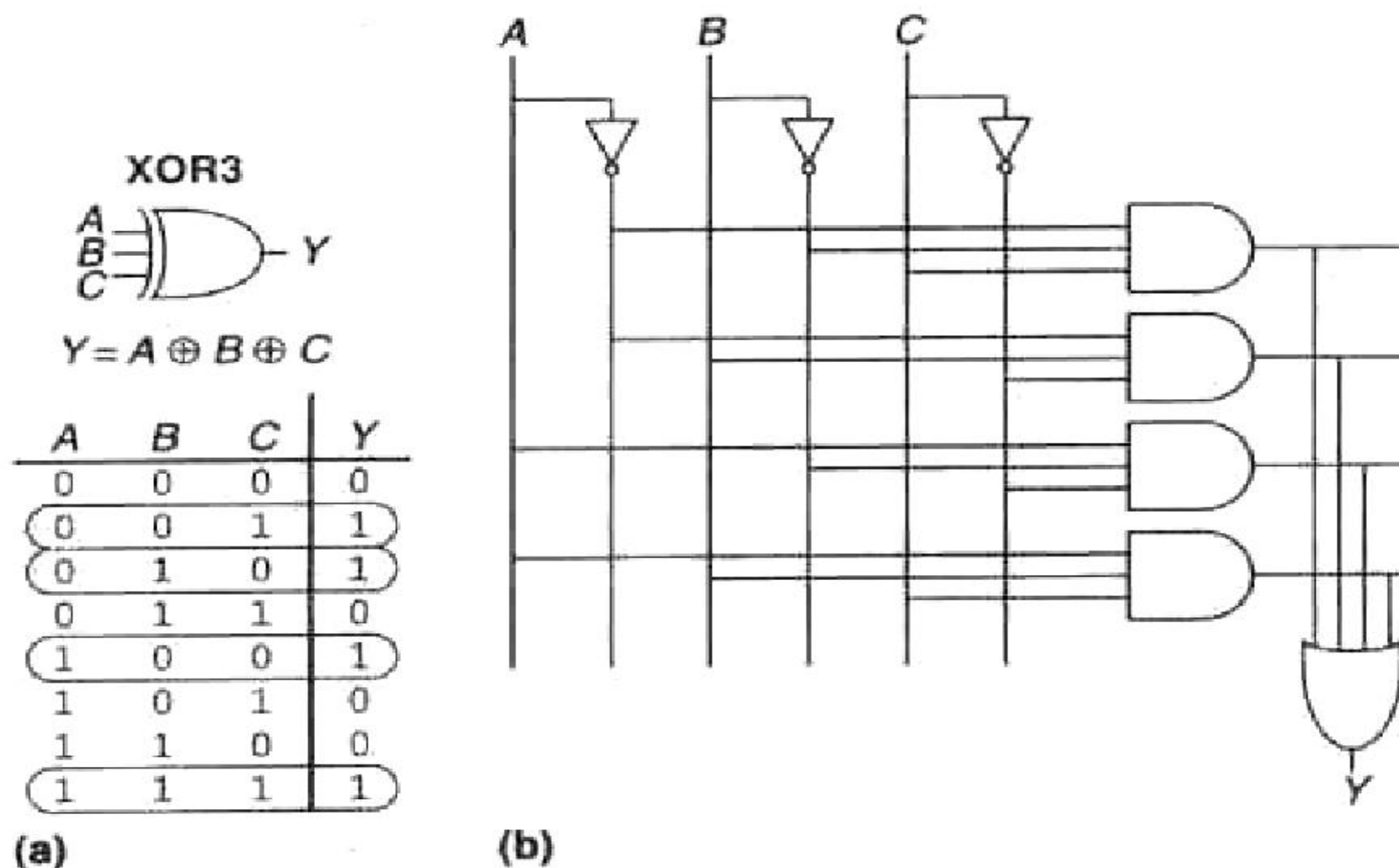


图2-30 3输入异或门：(a) 功能规范；(b) 两级逻辑实现

$$Y = \bar{A}\bar{B}C + \bar{A}B\bar{C} + A\bar{B}\bar{C} + ABC$$



图2-31 使用两个二输入异或门
构造一个三输入异或门

- a two level and-or logic implementation needs 128 8 input and-gate and 1 128 input or-gate.

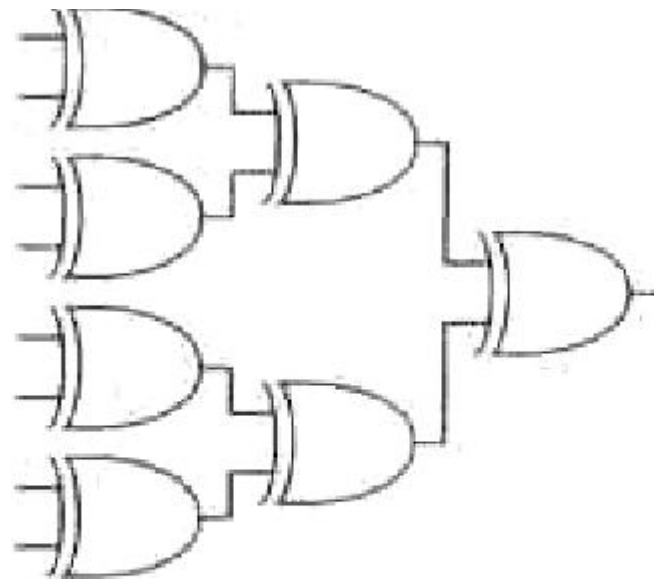


图2-32 用7个二输入异或门构造一个八输入异或门

7 two-input XOR gate can for a same circuit.

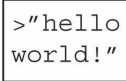
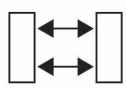
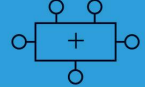
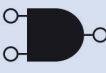
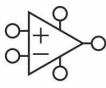
(*)



- Standard for Good?! (how to define Good?)
 - number of gate
 - running speed
 - time for design
 - cost
 - power consumption

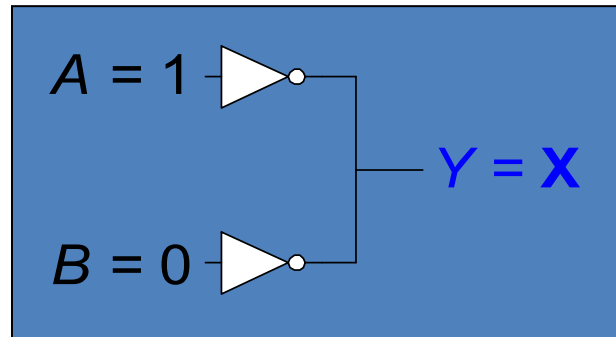
Chapter 2 :: Topics

- Introduction
- Boolean Equations
- Boolean Algebra
- From Logic to Gates
- **X's and Z's**
- Karnaugh Maps
- Combinational Building Blocks
- Timing

Application Software	
Operating Systems	
Architecture	
Micro-architecture	
Logic	
Digital Circuits	
Analog Circuits	
Devices	
Physics	

Unknown or Illegal: X

- Contention: circuit tries to drive output to 1 **and** 0
 - Actual value somewhere in between
 - Could be 0, 1, or in forbidden zone
 - Might change with voltage, temperature, time, noise
 - Often causes excessive power dissipation

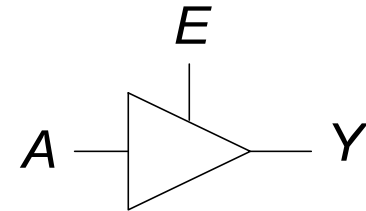


- **Warnings:**
 - Contention usually indicates a **bug**.
 - **X** is used for “don’t care” and contention - **look at the context to tell them apart**

Floating: Z

- Floating(浮空), high impedance(高阻), open, high Z
- Floating output might be 0, 1, or somewhere in between
 - A voltmeter won't indicate whether a node is floating

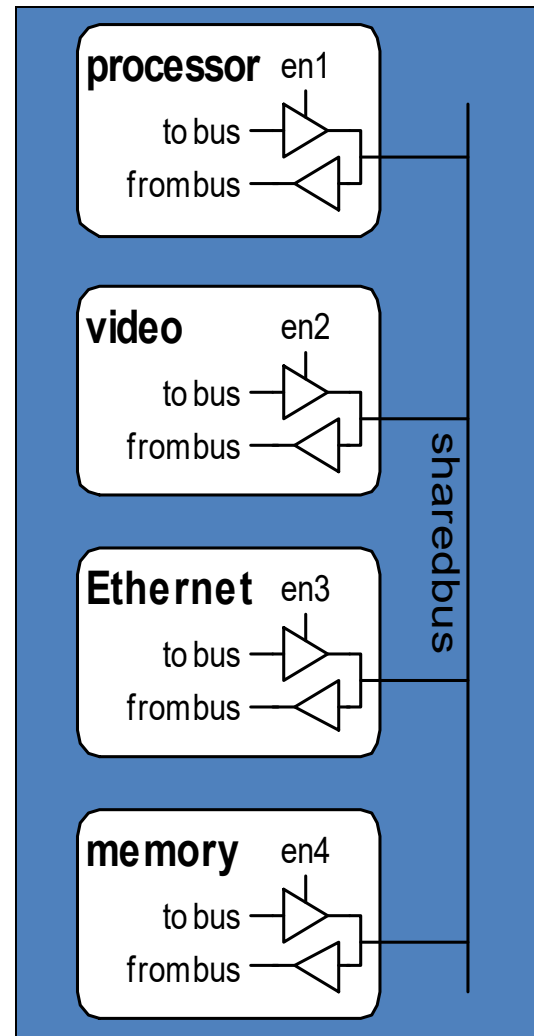
Tristate Buffer (三态缓冲器)



E	A	Y
0	0	Z
0	1	Z
1	0	0
1	1	1

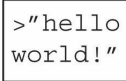
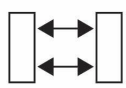
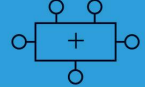
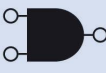
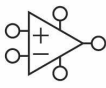
Tristate Busses

- Floating nodes are used in tristate busses
 - Many different drivers
 - Exactly one is active at once



Chapter 2 :: Topics

- Introduction
- Boolean Equations
- Boolean Algebra
- From Logic to Gates
- X's and Z's
- **Karnaugh Maps**
- Combinational Building Blocks
- Timing

Application Software	
Operating Systems	
Architecture	
Micro-architecture	
Logic	
Digital Circuits	
Analog Circuits	
Devices	
Physics	

- We could get different logic expressions with boolean algebra simplification or other methods.
- It is hard to learn and comprehend boolean algebra.
- Therefore, we need other effective and useful methods for simplification.
- Karnaugh map, K-map, is an alternative way for simplifying boolean expression, which was invented by Maurice Karnaugh in 1953(a member in Bell Lab)
- K-map is very appropriate for problems that containing up to 4 variables.
- More importantly, it is a visualization method for boolean expression simplification, which is more intuitive.

Karnaugh Maps (K-Maps)

- Boolean expressions can be minimized by combining terms, K-maps minimize equations graphically.
- Maurice Karnaugh, 1924–2022.
- Graduated with a bachelor's degree in physics from the City College of New York in 1948 and earned a Ph.D. in physics from Yale in 1952. Worked at Bell Labs and IBM from 1952 to 1993 and as a computer science professor at the Polytechnic University of New York from 1980 to 1999.

K-maps method

K-maps simplification

- K maps is an expression method for logic function. It is a grid that formed according to the relationship between minterms. Each small square/grid represents a minterm.

- $PA + P\bar{A} = P$

P	A	Y
0	0	0
0	1	0
1	0	1
1	1	1

		A	
		0	1
P	0	0	0
	1	1	1

		A	
		0	1
P	0	$\bar{P}\bar{A}$	$\bar{P}A$
	1	$P\bar{A}$	PA

- K maps can express the relationship between neighbor minterms graphically. Then we can simplify logic function by **merging neighbor minterms** iteratively.

K-maps method

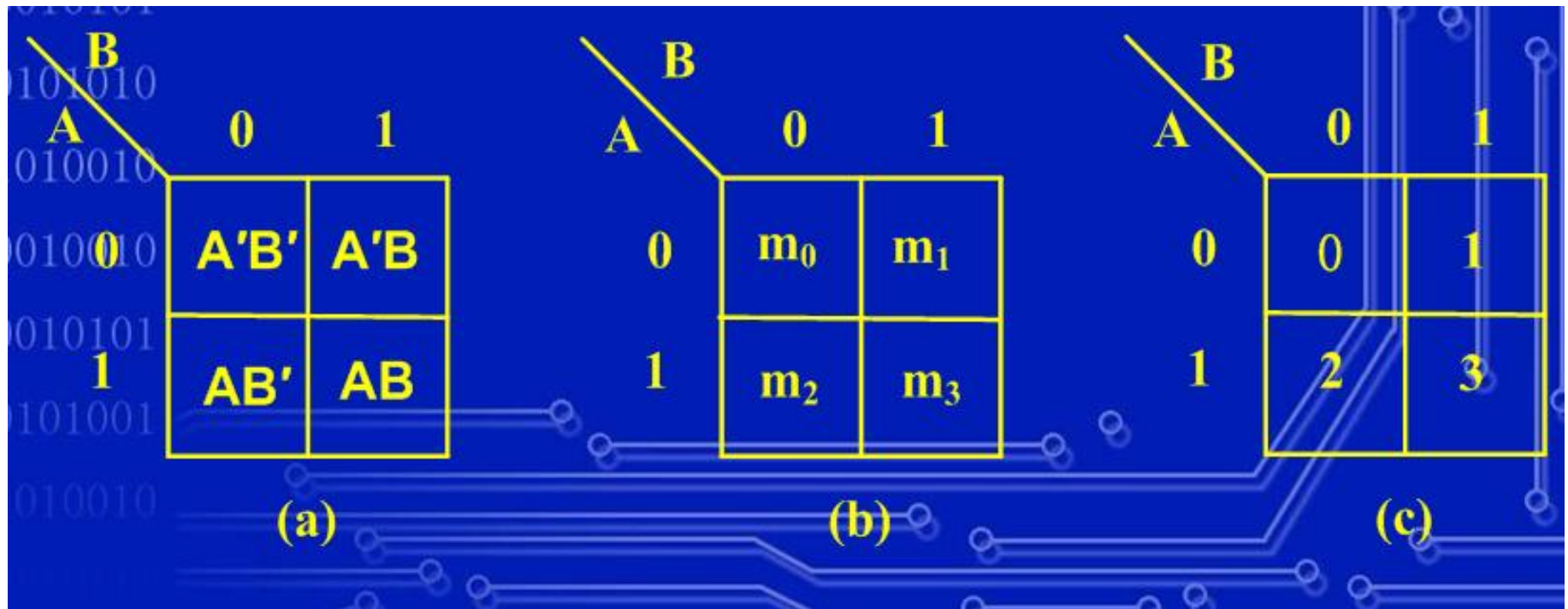
(1) composition of K-maps

- Using a table to represent all minterms of n variables, and make logical adjacent minterms that geometric adjacent. Then we get a K-maps for n variables minterms.
- For there a 2^n minterms for n variables, and each one correponds to a grid, then there are 2^n grids for n variable K-maps.

K-maps method

① K-maps for 2 variables

- 2 variables A and B form 4 minterms, they are represented with 4 neighbors grids.



K-maps method

② K-maps for 3 variables

$A \backslash BC$	$\bar{B}\bar{C}$	$\bar{B}C$	BC	$B\bar{C}$
	$\bar{A}\bar{B}\bar{C}$	$\bar{A}\bar{B}C$	$\bar{A}BC$	$\bar{A}B\bar{C}$
$\bar{A}$				
A				

每格标最小项

$A \backslash BC$	00	01	11	10
	m_0	m_1	m_3	m_2
0				
1				

每格标最小项编号

$A \backslash BC$	00	01	11	10
	000	001	011	010
0				
1				

每格标变量取值

$A \backslash BC$	00	01	11	10
	0	1	3	2
0				
1				

每格标最小项编号的简写

K-maps method

③ K-maps for 4 variables

AB \ CD				
	00	01	11	10
00	m_0	m_1	m_3	m_2
01	m_4	m_5	m_7	m_6
11	m_{12}	m_{13}	m_{15}	m_{14}
10	m_8	m_9	m_{11}	m_{10}

每格标最小项编号

AB \ CD				
	00	01	11	10
00	0000	0001	0011	0010
01	0100	0101	0111	0110
11	1100	1101	1111	1110
10	1000	1001	1011	1010

每格标变量取值

■ Note: order in serpentine arrangement

0 1 3 2 6 7 5 4 12 13 15 14 10 11 9 8

K-maps method

④ K-maps for 5 variables

$AB \backslash CDE$									
		000	001	011	010	110	111	101	100
00		0	1	3	2	6	7	5	4
01		8	9	11	10	14	15	13	12
11		24	25	27	26	30	31	29	28
10		16	17	19	18	22	23	21	20

逻辑相邻:

①相接 

②相对 

③相重 

■ Note : Rarely used K-maps to simplify boolean equation with more than 5 variables.

Karnaugh Maps (K-Maps)

A	B	C	Y
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	0

Y C	AB			
	00	01	11	10
0	1	0	0	0
1	1	0	0	0

Y C	AB			
	00	01	11	10
0	$\bar{A}\bar{B}\bar{C}$	$\bar{A}B\bar{C}$	$AB\bar{C}$	$A\bar{B}\bar{C}$
1	$\bar{A}\bar{B}C$	$\bar{A}BC$	ABC	$A\bar{B}C$

K-Map

- Circle 1's in adjacent squares
- In Boolean expression, include only literals whose true and complement form are *not* in the circle

A	B	C	Y
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	0

		AB			
C	Y	00	01	11	10
	0	1	0	0	0
1	1	1	0	0	0

$$Y = \overline{A}\overline{B}$$

3-Input K-Map

Y C \ AB		00	01	11	10
0	$\bar{A}\bar{B}\bar{C}$	$\bar{A}B\bar{C}$	$AB\bar{C}$	$A\bar{B}\bar{C}$	
1	$\bar{A}\bar{B}C$	$\bar{A}BC$	ABC	$A\bar{B}C$	

K-Map

Y C \ AB		00	01	11	10
0					
1					

3-Input K-Map

Y C \ AB		00	01	11	10
		$\bar{A}\bar{B}\bar{C}$	$\bar{A}B\bar{C}$	$AB\bar{C}$	$A\bar{B}\bar{C}$
0					
1					

K-Map

Y C \ AB		00	01	11	10
		0	1	1	0
0					
1					

$$Y = \bar{A}B + B\bar{C}$$

K-Map Definitions

- **Complement:** variable with a bar over it
 $\bar{A}, \bar{B}, \bar{C}$
- **Literal:** variable or its complement
 $\bar{A}, A, \bar{B}, B, C, \bar{C}$
- **Implicant:** product of literals
 $A\bar{B}C, \bar{A}C, BC$
- **Prime implicant:** implicant corresponding to the largest circle in a K-map

K-Map Rules

- Every 1 must be circled at least once
- Each circle must span a power of 2 (i.e. 1, 2, 4) squares in each direction
- Each circle must be as large as possible
- A circle may wrap around the edges
- A “don't care” (X) is circled only if it helps to minimize the equation

4-Input K-Map

<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>Y</i>
0	0	0	0	1
0	0	0	1	0
0	0	1	0	1
0	0	1	1	1
0	1	0	0	0
0	1	0	1	1
0	1	1	0	1
0	1	1	1	1
1	0	0	0	1
1	0	0	1	1
1	0	1	0	1
1	0	1	1	0
1	1	0	0	0
1	1	0	1	0
1	1	1	0	0
1	1	1	1	0

<i>Y</i>				
<i>CD</i>	<i>AB</i>			
	00	01	11	10
00				
01				
11				
10				

4-Input K-Map

<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>Y</i>
0	0	0	0	1
0	0	0	1	0
0	0	1	0	1
0	0	1	1	1
0	1	0	0	0
0	1	0	1	1
0	1	1	0	1
0	1	1	1	1
1	0	0	0	1
1	0	0	1	1
1	0	1	0	1
1	0	1	1	0
1	1	0	0	0
1	1	0	1	0
1	1	1	0	0
1	1	1	1	0

		<i>Y</i>			
		<i>AB</i>			
<i>CD</i>		00	01	11	10
	00	1	0	0	1
	01	0	1	0	1
	11	1	1	0	0
	10	1	1	0	1

4-Input K-Map

A	B	C	D	Y
0	0	0	0	1
0	0	0	1	0
0	0	1	0	1
0	0	1	1	1
0	1	0	0	0
0	1	0	1	1
0	1	1	0	1
0	1	1	1	1
1	0	0	0	1
1	0	0	1	1
1	0	1	0	1
1	0	1	1	0
1	1	0	0	0
1	1	0	1	0
1	1	1	0	0
1	1	1	1	0

Y		AB			
		00	01	11	10
CD	00	1	0	0	1
	01	0	1	0	1
	11	1	1	0	0
	10	1	1	0	1

$Y = \bar{A}\bar{C} + \bar{A}BD + A\bar{B}\bar{C} + \bar{B}D$

K-Maps with Don't Cares

<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>Y</i>
0	0	0	0	1
0	0	0	1	0
0	0	1	0	1
0	0	1	1	1
0	1	0	0	0
0	1	0	1	X
0	1	1	0	1
0	1	1	1	1
1	0	0	0	1
1	0	0	1	1
1	0	1	0	X
1	0	1	1	X
1	1	0	0	X
1	1	0	1	X
1	1	1	0	X
1	1	1	1	X

		<i>Y</i>			
		<i>AB</i>			
<i>CD</i>		00	01	11	10
	00				
	01				
	11				
	10				

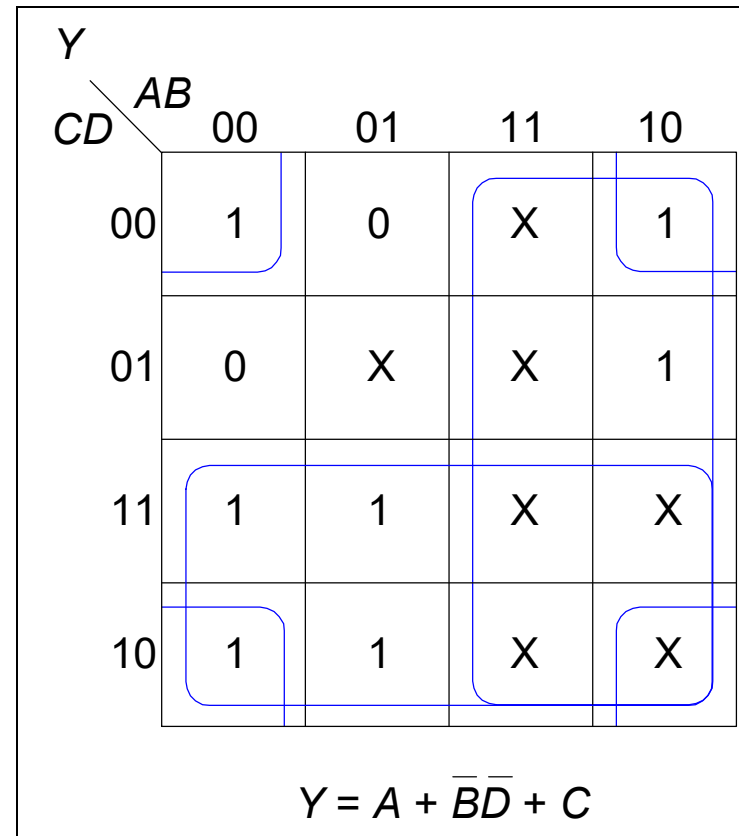
K-Maps with Don't Cares

A	B	C	D	Y
0	0	0	0	1
0	0	0	1	0
0	0	1	0	1
0	0	1	1	1
0	1	0	0	0
0	1	0	1	X
0	1	1	0	1
0	1	1	1	1
1	0	0	0	1
1	0	0	1	1
1	0	1	0	X
1	0	1	1	X
1	1	0	0	X
1	1	0	1	X
1	1	1	0	X
1	1	1	1	X

		Y			
		AB			
CD		00	01	11	10
	00	1	0	X	1
	01	0	X	X	1
	11	1	1	X	X
	10	1	1	X	X

K-Maps with Don't Cares

A	B	C	D	Y
0	0	0	0	1
0	0	0	1	0
0	0	1	0	1
0	0	1	1	1
0	1	0	0	0
0	1	0	1	X
0	1	1	0	1
0	1	1	1	1
1	0	0	0	1
1	0	0	1	1
1	0	1	0	X
1	0	1	1	X
1	1	0	0	X
1	1	0	1	X
1	1	1	0	X
1	1	1	1	X



K-maps for logic function simplification

- 1 step:
make sure grid with '1' and their neighbors.
3 types neighbors: adjacency, opposite and coincide
2 step:
circle '1' grids with neighbor relationship.
number of grids in a circle should be 2^n
minimal for number of circles, maximal for number of grids in a circle.

K-maps for logic function simplification

$AB \backslash CD$	00	01	11	10
00	0	1	3	2
01	4	5	7	6
11	12	13	15	14
10	8	9	11	10

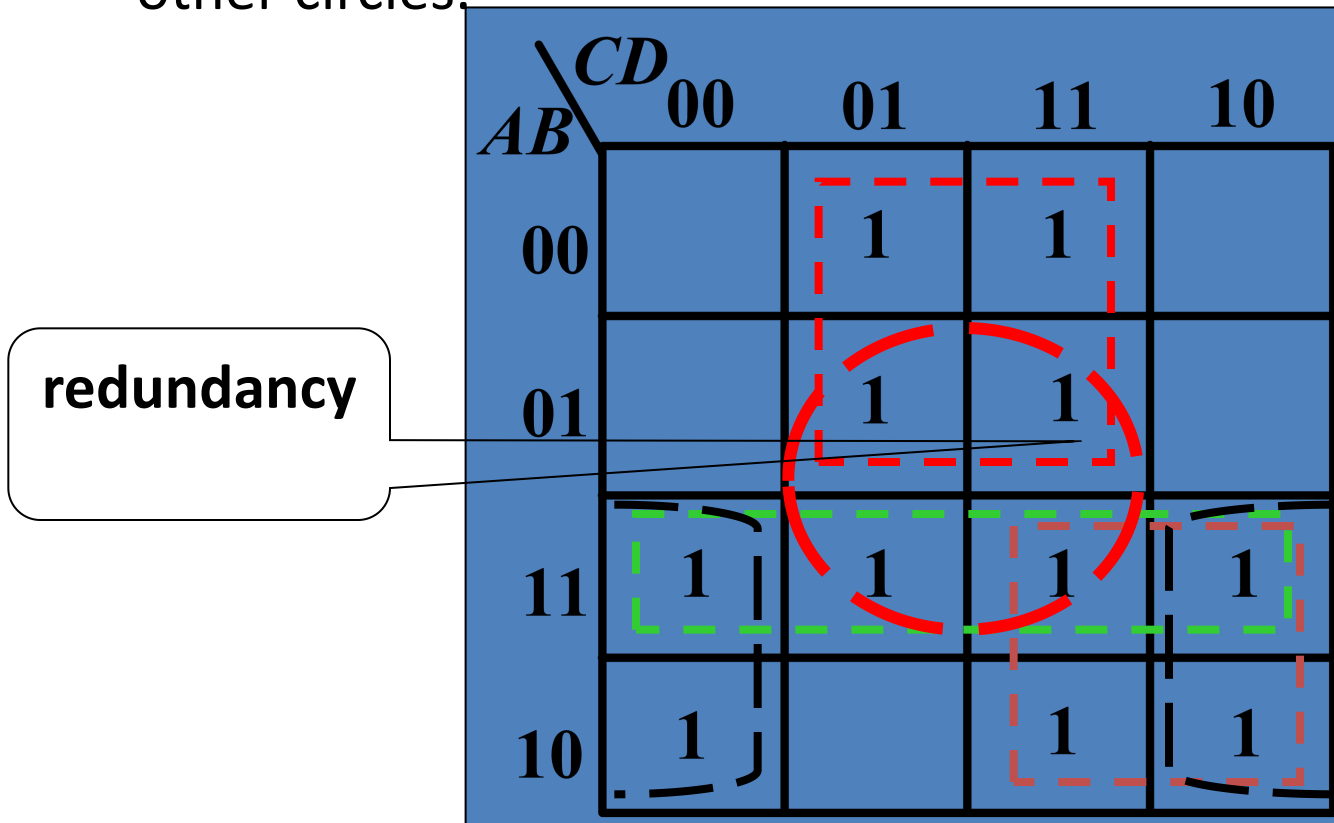
$AB \backslash CD$	00	01	11	10
00	0	1	3	2
01	4	5	7	6
11	12	13	15	14
10	8	9	11	10

$AB \backslash CD$	00	01	11	10
00	0	1	3	2
01	4	5	7	6
11	12	13	15	14
10	8	9	11	10

$AB \backslash CD$	00	01	11	10
00	0	1	3	2
01	4	5	7	6
11	12	13	15	14
10	8	9	11	10

K-maps for logic function simplification

- grid with '1' can be included in many circles.
- In each circle, there is at least one '1' grid that does not exist in other circles.



K-maps for logic function simplification

- If there is '1' grid that can not be included in a circle, then it can not be simplified, ex m_8 .

$\backslash CD$					
		00	01	11	10
AB	00	0	1	1	0
	01	0	1	1	0
	11	0	1	1	0
	10	1	0	0	0

K-maps for logic function simplification

- 3 step: simplified logic function
 - based on K-maps
 - there may be many ways to draw circles in K-maps, hence the simplified logic functions may not be unified.

K-maps for logic function simplification

【ex1】 Using k-maps to simplify $F=AC' + A'C + BC' + B'C$

$A \backslash BC$					
		00	01	11	10
0	0	0	1	1	1
1	1	1	1	0	1

$$F=AB' + A'C + BC'$$

$A \backslash BC$					
		00	01	11	10
0	0	0	1	1	1
1	1	1	1	0	1

$$F=AC' + B'C + A'B$$

K-maps for logic function simplification

【ex2】 Using k-maps to simplify

$$F=ABC+ABD+AC'D+C'D'+AB'C+A'CD'$$

$$F=A+D'$$

$AB \backslash CD$	00	01	11	10
00	1	0	0	1
01	1	0	0	1
11	1	1	1	1
10	1	1	1	1

The K-map is a 4x4 grid with rows labeled AB (00, 01, 11, 10) and columns labeled CD (00, 01, 11, 10). The cells contain values 1 or 0. Red dashed lines indicate groupings: a horizontal group of four 1s in the row AB=11, a horizontal group of four 1s in the row AB=10, a vertical group of four 1s in the column CD=00, and a vertical group of four 1s in the column CD=10. Solid black lines also indicate groupings: a horizontal group of two 1s in the row AB=00, a horizontal group of two 1s in the row AB=01, a vertical group of two 1s in the column CD=00, and a vertical group of two 1s in the column CD=10.

K-maps for logic function simplification

(5) simplification with 'don't care item'

- don't care item is represented with “ Φ ” or “ $\times$ ” in K-maps ($\sum d(\dots)$), and it can be 0 or 1.

K-maps for logic function simplification

【ex 3】 Using k-maps to simplify $F = \sum m(0,1,2,4) + \sum d(5,6)$

BC		00	01	11	10
A	0	1	1	0	1
	1	1	×	0	×

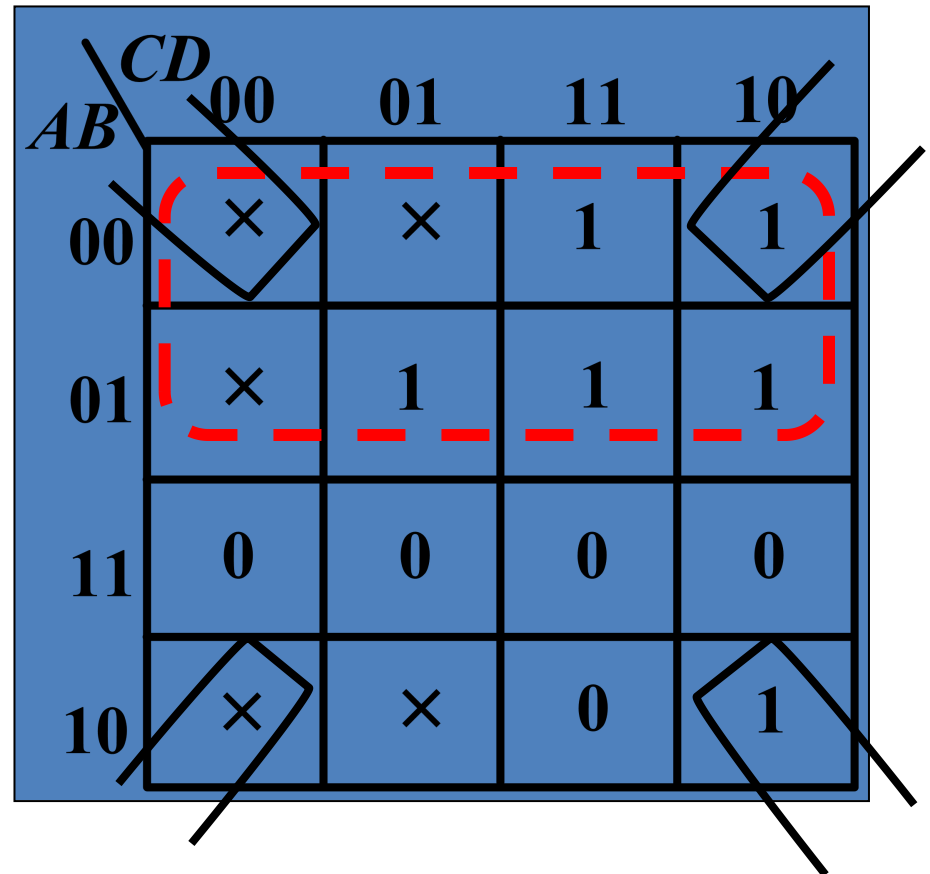
$$F = B' + C'$$

K-maps for logic function simplification

【ex 4】 Using k-maps to simplify

$$F = \sum m(3, 5, 6, 7, 10) + \sum d(0, 1, 2, 4, 8, 9)$$

$$F = A' + B'D'$$



ex: logic function simplification

$$F(A,B,C,D)=\sum m(15,13,10,6,4)+\sum d(8,7,5,2,1,0)$$

■ no don't care item

$$F = A'BD' + ABD + AB'CD'$$

■ don't care item

$$F = A'D' + BD + B'D'$$

$AB \backslash CD$	00	01	11	10
00	×	×	0	×
01	1	×	×	1
11	0	1	1	0
10	×	0	0	1

K-maps for logic function simplification

- ex:

$$F = \sum(1, 3, 5, 9) + \sum_d(7, 11, 13)$$

- an:
- ① draw K-maps
 - ② merge minterm
 - ③ simplified function

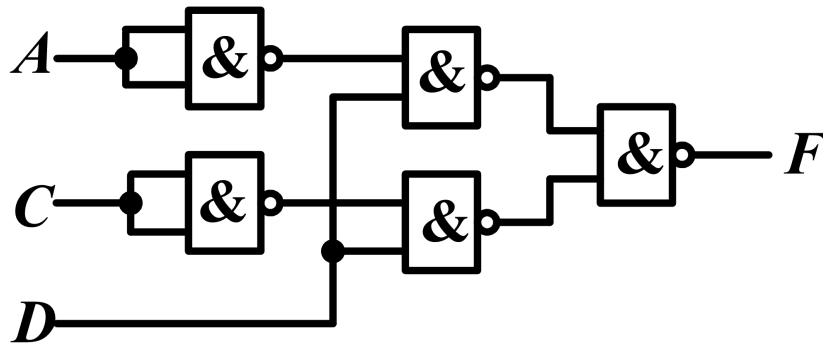
$$\begin{aligned}
 F &= A'D + C'D \\
 &= (A'D + C'D)'' \\
 &= ((A'D)' \cdot (C'D)')' \\
 F &= A'D + C'D \\
 &= (A' + C')D \\
 &= ((AC)' \cdot D)''
 \end{aligned}$$

$AB \backslash CD$					
		00	01	11	10
AB	00		1	1	
	01		1	×	
	11		×		
	10		1	×	

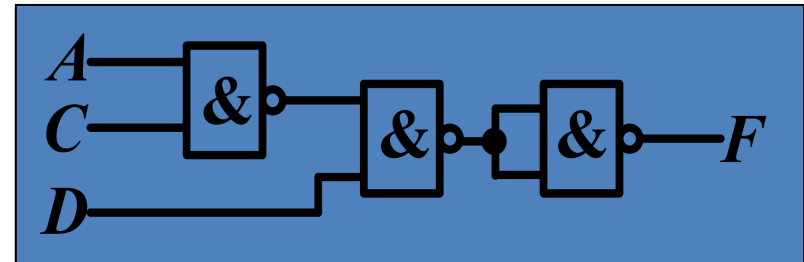
K-maps for logic function simplification

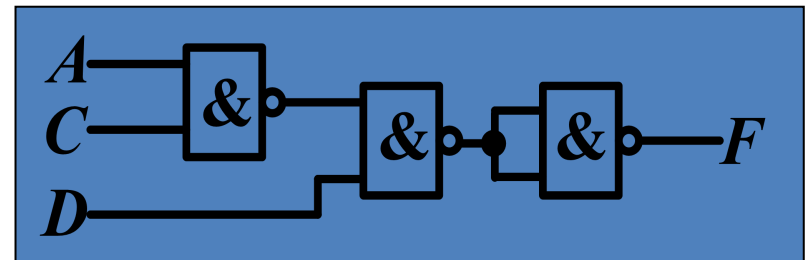
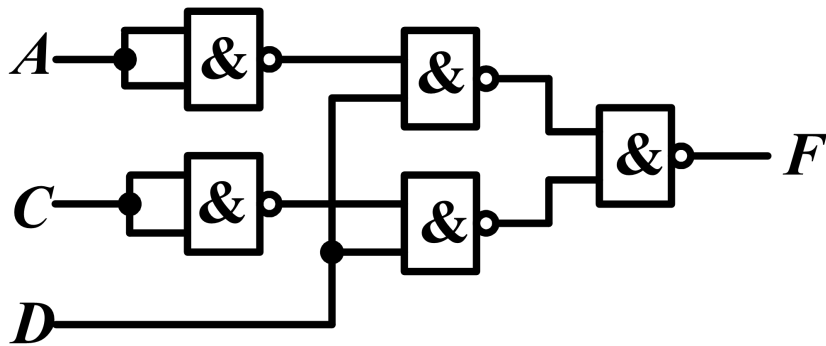
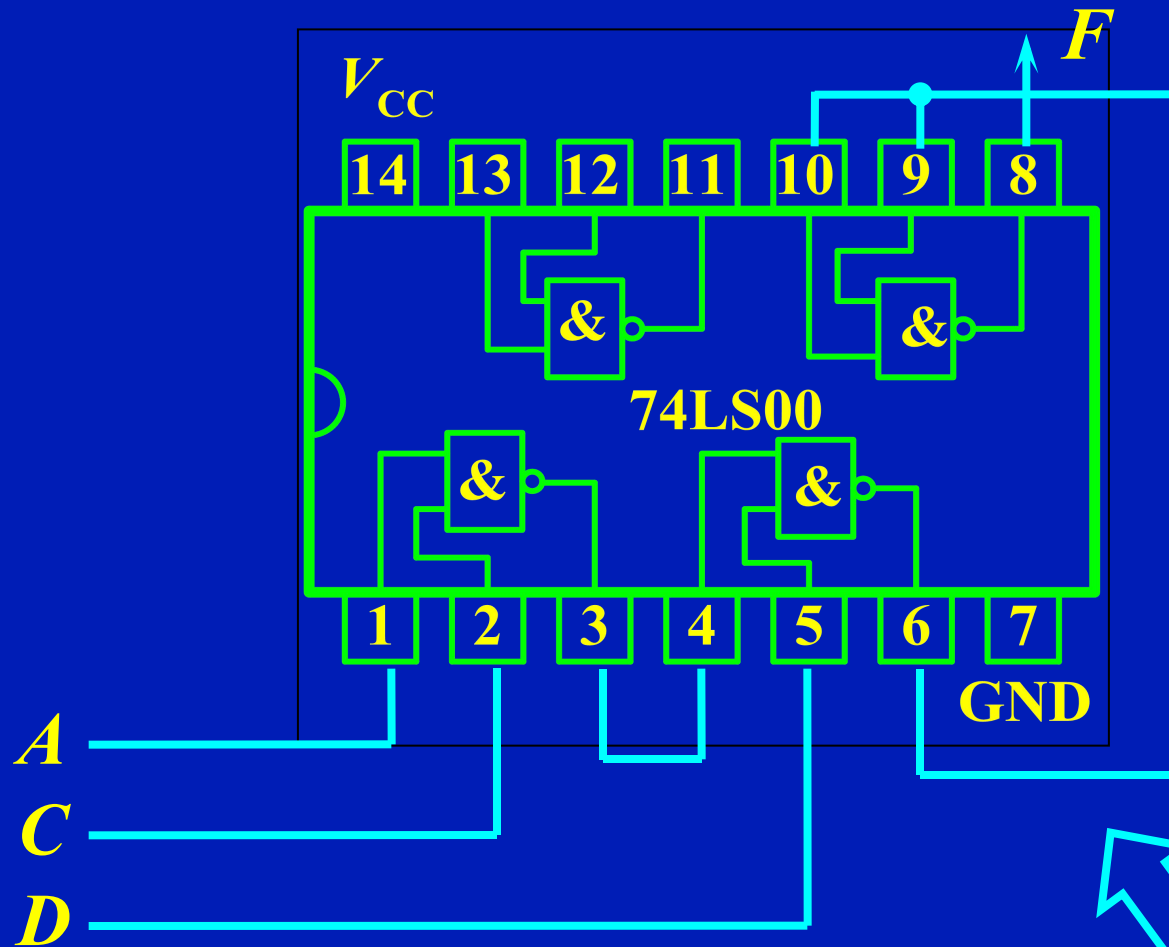
$$F = \sum(1, 3, 5, 9) + \sum_d(7, 11, 13)$$

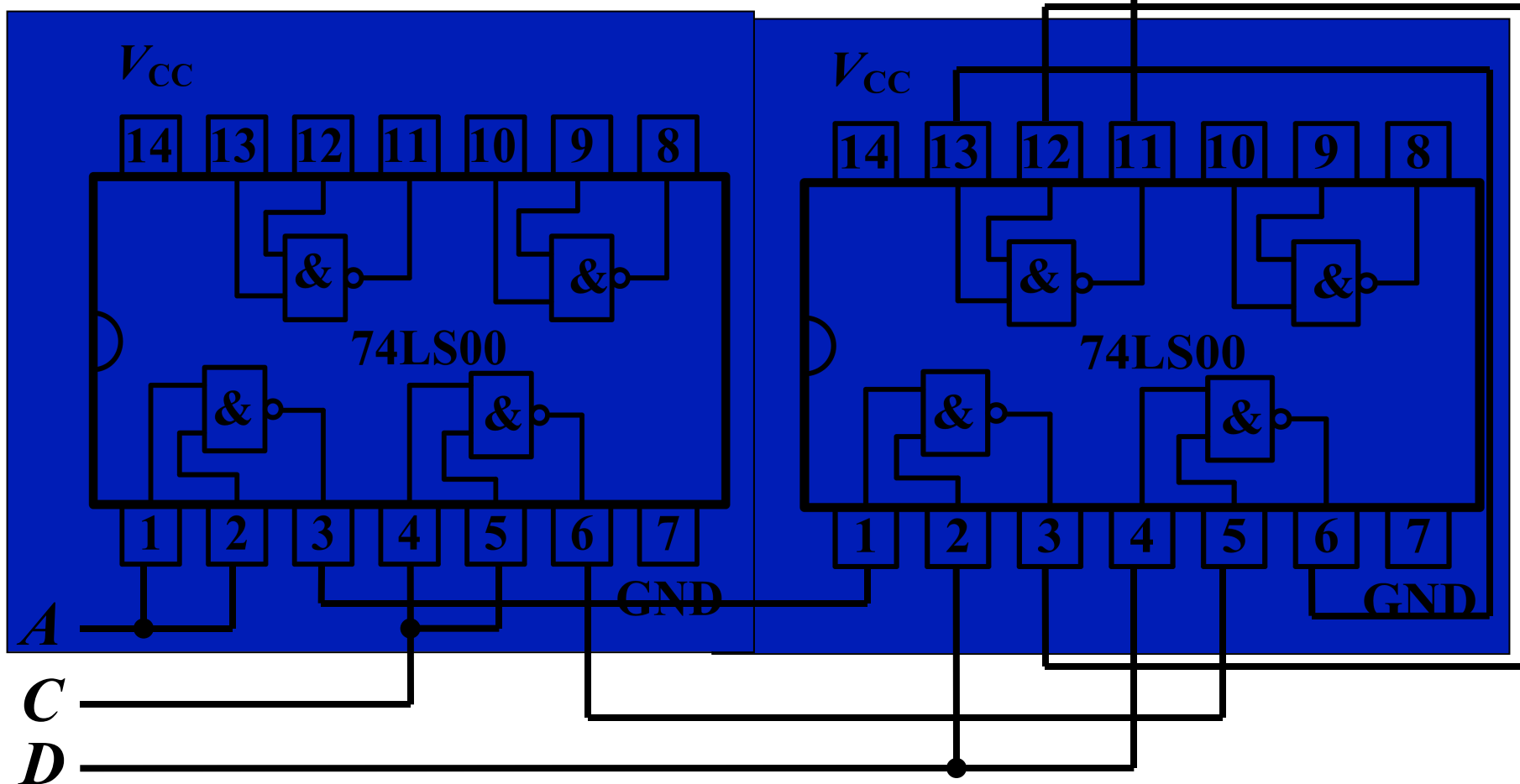
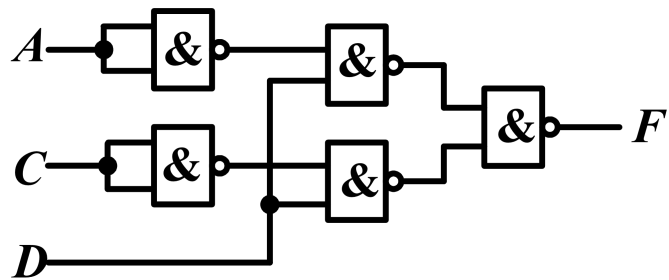
$$\begin{aligned} F &= A'D + C'D \\ &= (A'D + C'D)'' \\ &= ((A'D)' \cdot (C'D)')' \end{aligned}$$



$$\begin{aligned} F &= A'D + C'D \\ &= (A' + C')D \\ &= ((AC)' \cdot D)'' \end{aligned}$$







K-maps for logic function simplification

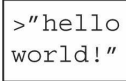
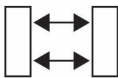
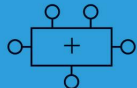
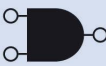
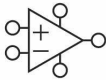
- logic function generalized from basic problem may not be simplest, so we should simplify it to get a concise one.
- Moreover, for a simplest logic function, it may not be suit to the given(available) logic gate. The practical facts require us to modify the simplest logic function.

K-maps for logic function simplification

- two main methods
 - using boolean algebra
 - using k-maps

Chapter 2 :: Topics

- Introduction
- Boolean Equations
- Boolean Algebra
- From Logic to Gates
- Combinational Logic
- X's and Z's, Oh My
- Karnaugh Maps
- **Combinational Building Blocks**
- Timing

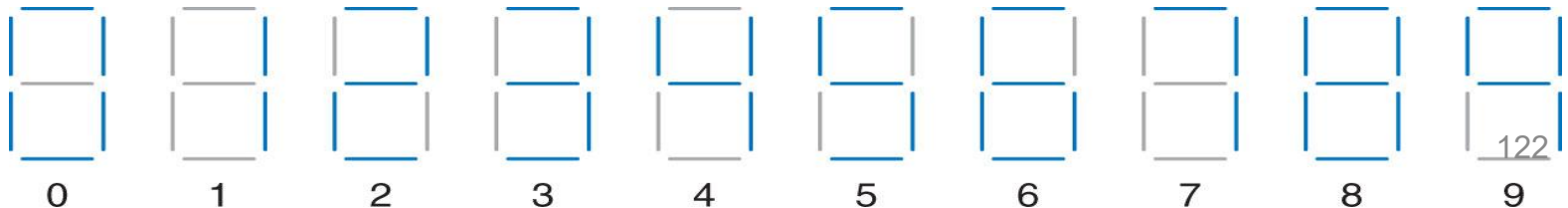
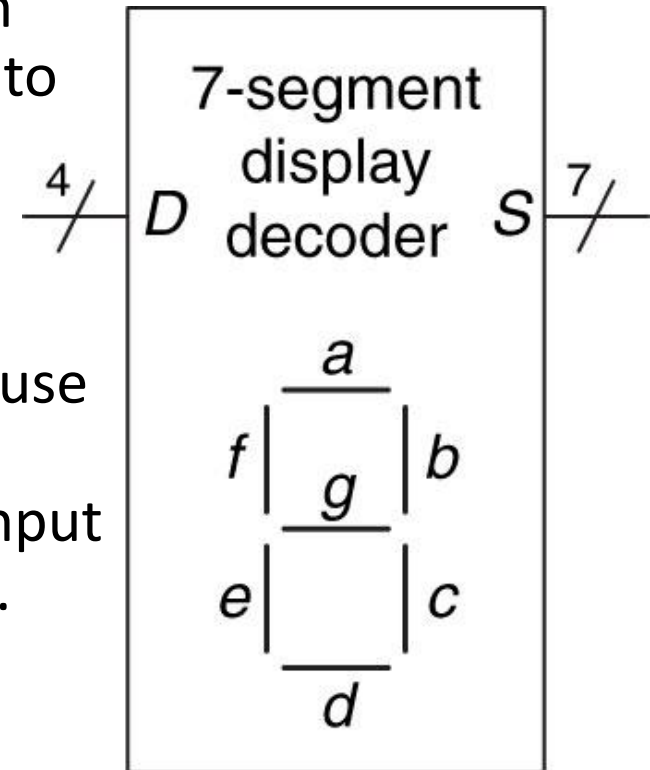
Application Software	
Operating Systems	
Architecture	
Micro-architecture	
Logic	
Digital Circuits	
Analog Circuits	
Devices	
Physics	

Combinational Building Blocks

- **SEVEN-SEGMENT DISPLAY DECODER**
- **Multiplexers**
- **Decoders**

SEVEN-SEGMENT DISPLAY DECODER

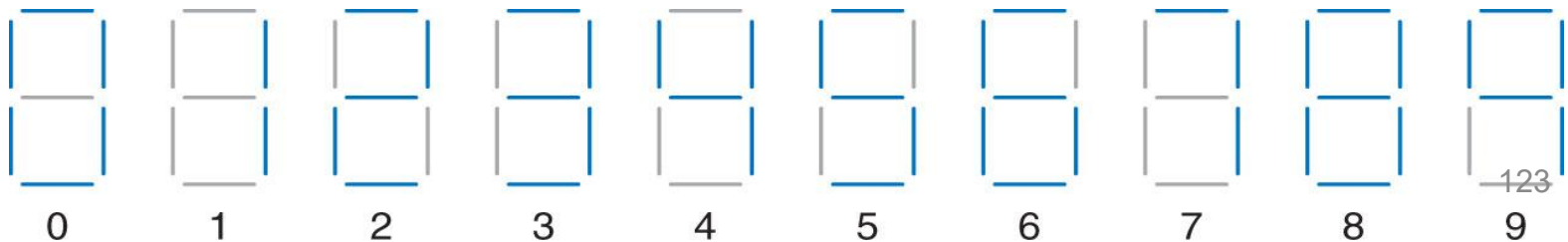
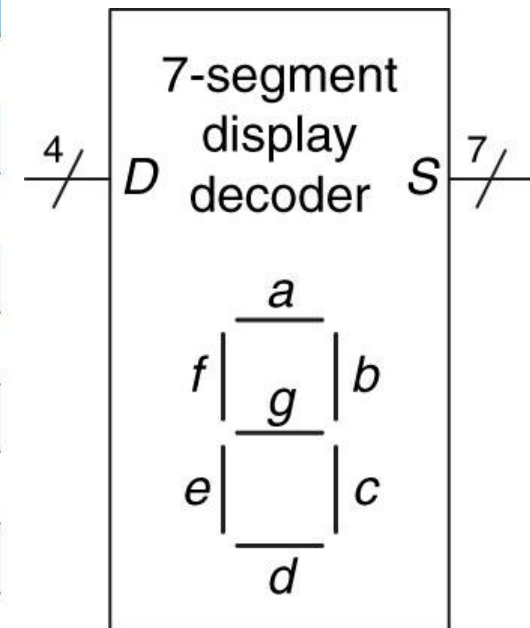
- A seven-segment display decoder takes a 4-bit data input, $D3:0$, and produces seven outputs to control light-emitting diodes to display a digit from 0 to 9.
- The seven outputs are often called segments *a through g*, or $Sa-Sg$.
- Write a truth table for the outputs, and use K-maps to find Boolean equations for outputs Sa and Sb . Assume that illegal input values (10–15) produce a blank readout.



SEVEN-SEGMENT DISPLAY DECODER

Table 2.6 Seven-segment display decoder truth table

$D_{3:0}$	S_a	S_b	S_c	S_d	S_e	S_f	S_g
0000	1	1	1	1	1	1	0
0001	0	1	1	0	0	0	0
0010	1	1	0	1	1	0	1
0011	1	1	1	1	0	0	1
0100	0	1	1	0	0	1	1
0101	1	0	1	1	0	1	1
0110	1	0	1	1	1	1	1
0111	1	1	1	0	0	0	0
1000	1	1	1	1	1	1	1
1001	1	1	1	0	0	1	1
others	0	0	0	0	0	0	0



SEVEN-SEGMENT DISPLAY DECODER

Table 2.6 Seven-seg

$D_{3:0}$	S_a	S_b
0000	1	1
0001	0	1
0010	1	1
0011	1	1
0100	0	1
0101	1	0
0110	1	0
0111	1	1
1000	1	1
1001	1	1
others	0	0

		$D_{3:2}$			
		00	01	11	10
$D_{1:0}$	00	1	0	0	1
	01	0	1	0	1
	11	1	1	0	0
	10	1	1	0	0

SEVEN-SEGMENT DISPLAY DECODER

S_a		$D_{3:2}$			
$D_{1:0}$		00	01	11	10
00		1	0	0	1
01		0	1	0	1
11		1	1	0	0
10		1	1	0	0

S_a		$D_{3:2}$			
$D_{1:0}$		00	01	11	10
00		1	0	0	1
01		0	1	0	1
11		1	1	0	0
10		1	1	0	0

$\bar{D}_2 \bar{D}_1 \bar{D}_0$ (points to cell 00,00)
 $\bar{D}_3 \bar{D}_2 D_0$ (points to cell 01,01)
 $\bar{D}_3 D_1$ (points to cell 11,10)
 $D_3 \bar{D}_2 \bar{D}_1$ (points to cell 10,01)

$$S_a = \bar{D}_3 D_1 + \bar{D}_3 \bar{D}_2 D_0 + D_3 \bar{D}_2 \bar{D}_1 + \bar{D}_2 \bar{D}_1 \bar{D}_0$$

SEVEN-SEGMENT DISPLAY DECODER

S_a		$D_{3:2}$				
		$D_{1:0}$	00	01	11	10
00		00	1	0	0	1
01		01	0	1	0	1
11		11	1	1	0	0
10		10	1	1	0	0

$$S_a = \bar{D}_3 D_1 + \bar{D}_3 D_2 D_0 + D_3 \bar{D}_2 \bar{D}_1 + \bar{D}_3 \bar{D}_2 \bar{D}_0$$

S_a		$D_{3:2}$			
		$D_{1:0}$	00	01	11
00		1	0	0	1
01		0	1	0	1
11		1	1	0	0
10		1	1	0	0

$\bar{D}_2 \bar{D}_1 \bar{D}_0$

$\bar{D}_3 D_2 D_0$

$D_3 \bar{D}_2 \bar{D}_1$

$\bar{D}_3 D_1$

$$S_a = \bar{D}_3 D_1 + \bar{D}_3 D_2 D_0 + D_3 \bar{D}_2 \bar{D}_1 + \bar{D}_2 \bar{D}_1 \bar{D}_0$$

SEVEN-SEGMENT DISPLAY DECODER

- with don't cares (we don't care about the output values for illegal input values of 10 to 15.)

S_a	$D_{3:2}$	00	01	11	10
$D_{1:0}$	00	1	0	0	1
	01	0	1	0	1
	11	1	1	0	0
	10	1	1	0	0

S_a	$D_{3:2}$	00	01	11	10
$D_{1:0}$	00	1	0	X	1
	01	0	1	X	1
	11	1	1	X	X
	10	1	1	X	X

$$S_a = D_3 + D_2 D_0 + \bar{D}_2 \bar{D}_0 + D_1$$

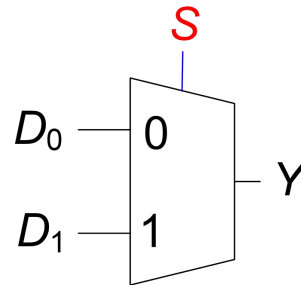
S_b		$D_{3:2}$			
		00	01	11	10
$D_{1:0}$	00	1	1	0	1
	01	1	0	0	1
	11	1	1	0	0
	10	1	0	0	0

S_b		$D_{3:2}$			
$D_{1:0}$		00	01	11	10
00	1	1	X	1	
01	1	0	X	1	
11	1	1	X	X	
10	1	0	X	X	

$$S_b = \bar{D}_2 + D_1 D_0 + \bar{D}_1 \bar{D}_0$$

Multiplexer (Mux)

- Selects between one of N inputs to connect to output
- $\log_2 N$ -bit **select input** – control input
- Example: **2:1 Mux**



S	D_1	D_0	Y
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

S	Y
0	D_0
1	D_1

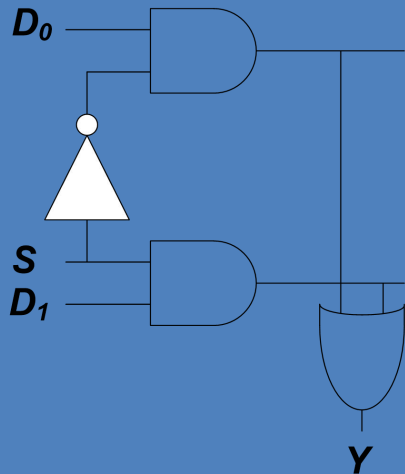
Multiplexer Implementations

- **Logic gates**

- Sum-of-products form

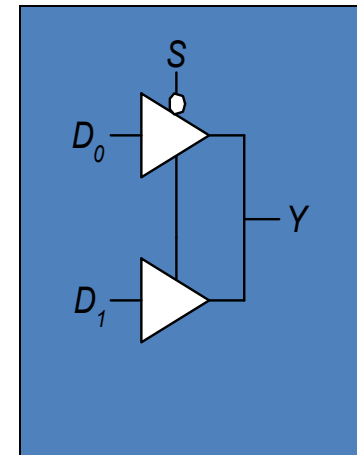
Y S	D ₀ D ₁		00	01	11	10
	0	1	0	0	1	1
1	0	1	0	1	1	0

$$Y = D_0\bar{S} + D_1S$$



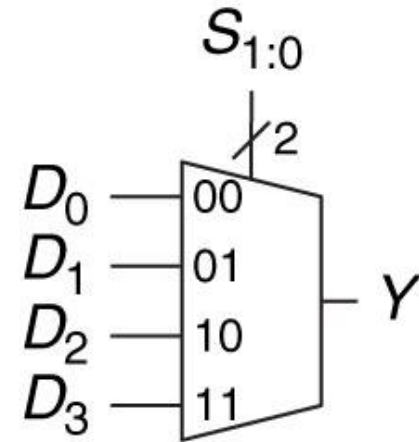
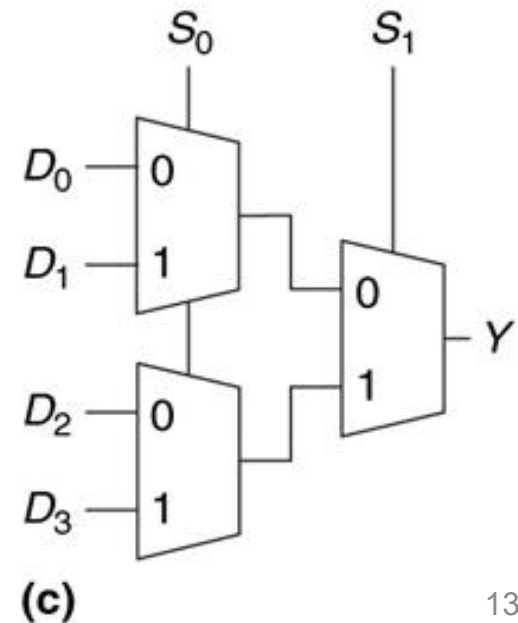
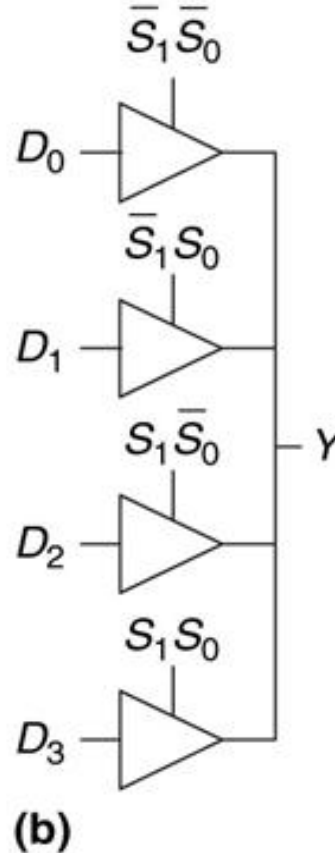
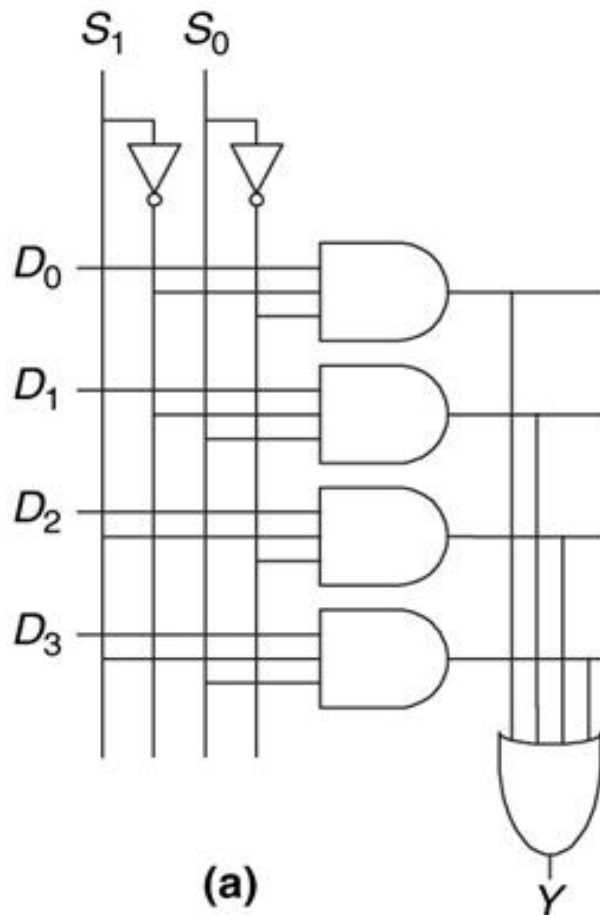
- **Tristates**

- For an N-input mux, use N tristates
- Turn on exactly one to select the appropriate input



Wider Multiplexers

- 4:1 multiplexer
 - has four data inputs and one output

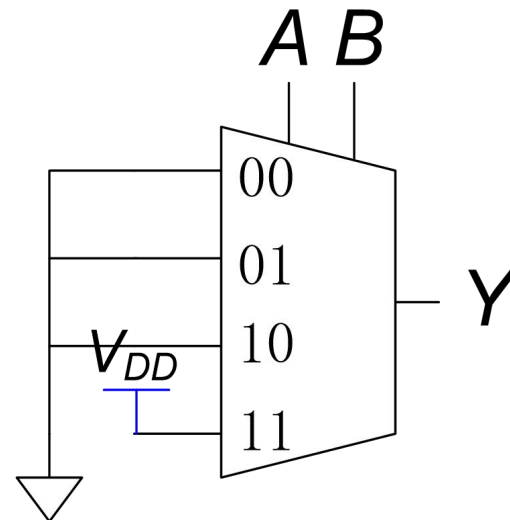


Logic using Multiplexers

- Using the mux as a lookup table

<i>A</i>	<i>B</i>	<i>Y</i>
0	0	0
0	1	0
1	0	0
1	1	1

$$Y = AB$$



Example 2.12 LOGIC WITH MULTIPLEXERS

- Alyssa P. Hacker needs to implement the function $Y = A\bar{B} + \bar{B}\bar{C} + \bar{A}BC$ to finish her senior project, but when she looks in her lab kit, the only part she has left is an 8:1 multiplexer.
- How does she implement the function?

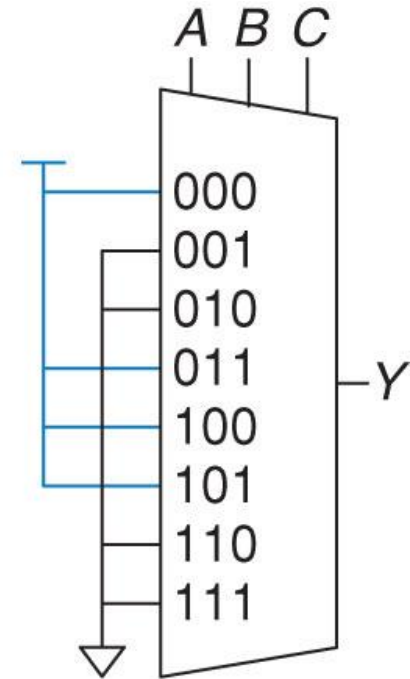
Example 2.12 LOGIC WITH MULTIPLEXERS

- Solution: Figure shows Alyssa's implementation using a single 8:1 multiplexer.
- The multiplexer acts as a lookup table where each row in the truth table corresponds to a multiplexer input.

A	B	C	Y
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	0

$$Y = A\bar{B} + \bar{B}\bar{C} + \bar{A}BC$$

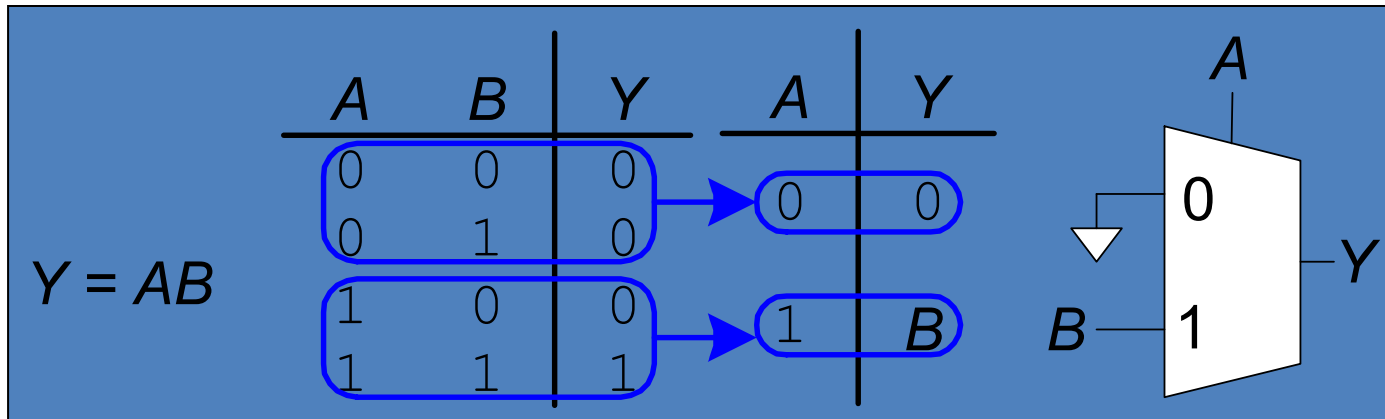
(a)



(b)

Logic using Multiplexers

- Reducing the size of the mux



Logic using Multiplexers

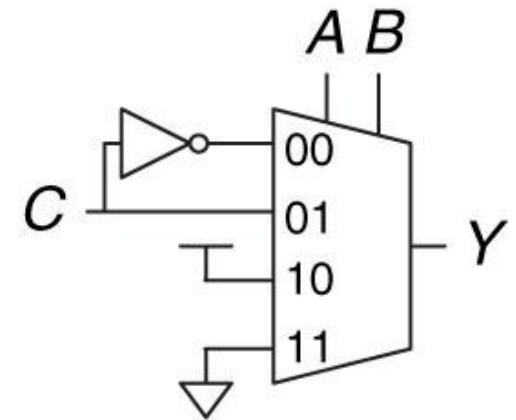
- Reducing the size of the mux

<i>A</i>	<i>B</i>	<i>C</i>	<i>Y</i>
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	0

(a)

<i>A</i>	<i>B</i>	<i>Y</i>
0	0	$\overline{C}$
0	1	<i>C</i>
1	0	1
1	1	0

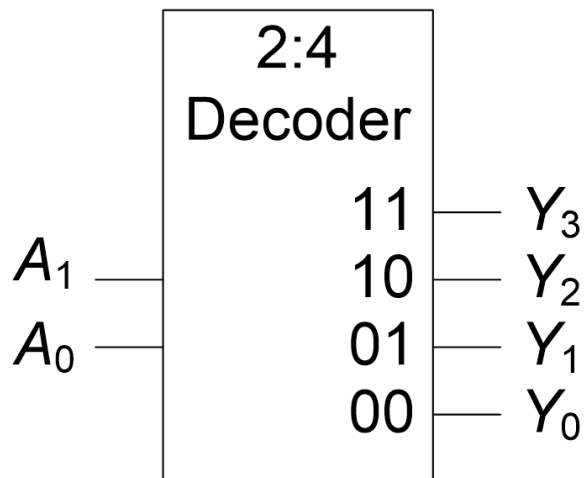
(b)



(c)

Decoders

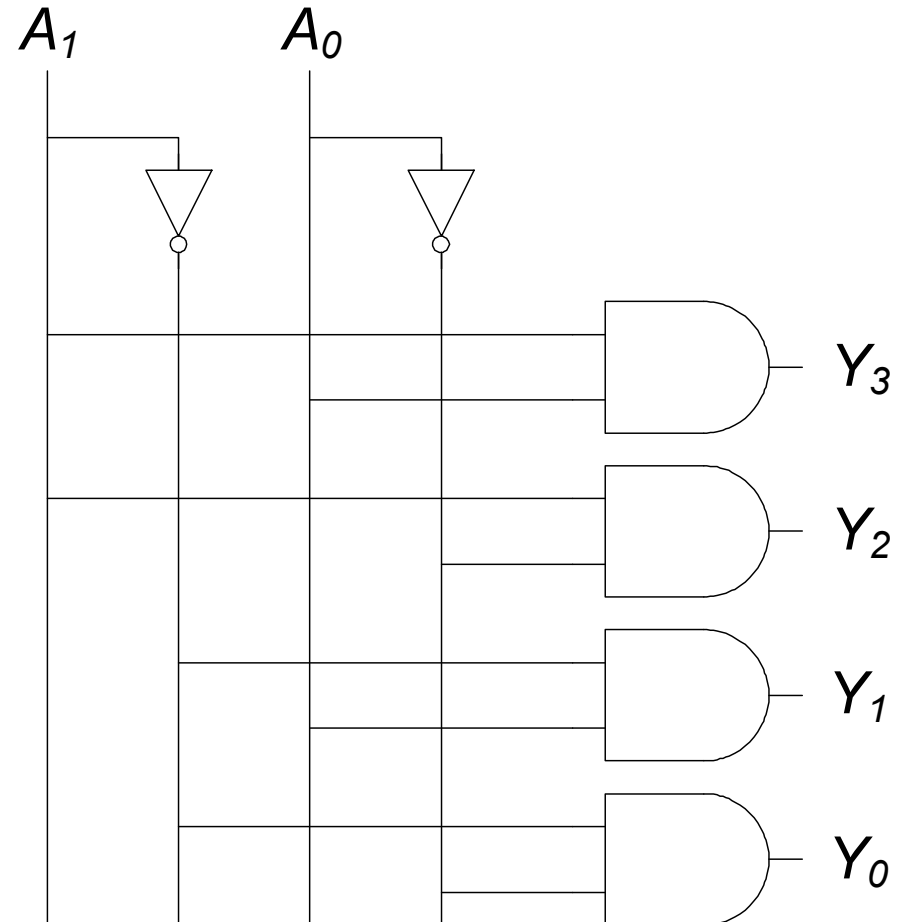
- N inputs, 2^N outputs
- **One-hot** outputs: only one output HIGH at once



A_1	A_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	1
0	1	0	0	1	0
1	0	0	1	0	0
1	1	1	0	0	0

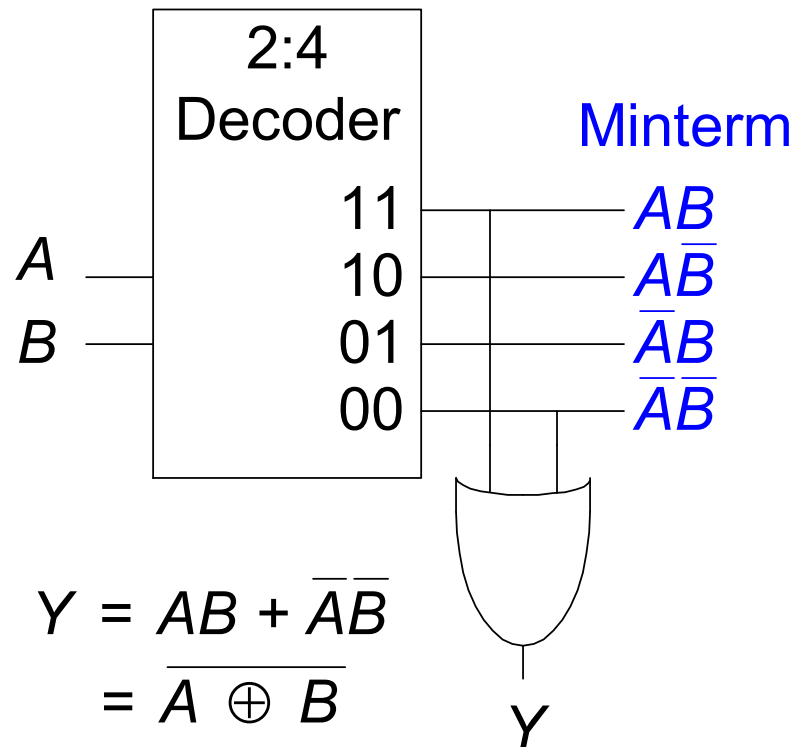
Decoder Implementation

A_1	A_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	1
0	1	0	0	1	0
1	0	0	1	0	0
1	1	1	0	0	0



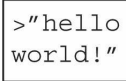
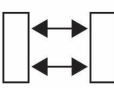
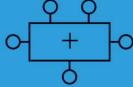
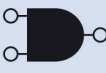
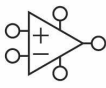
Logic Using Decoders

- OR minterms



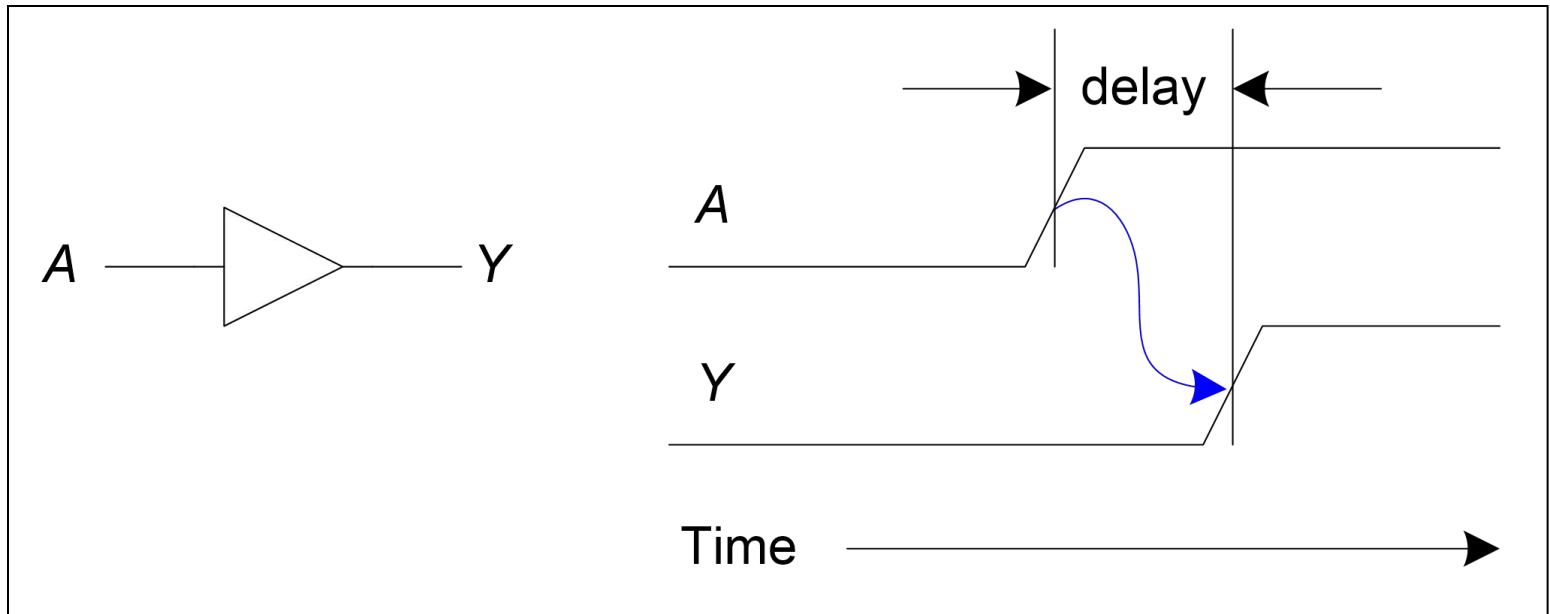
Chapter 2 :: Topics

- Introduction
- Boolean Equations
- Boolean Algebra
- From Logic to Gates
- Combinational Logic
- X's and Z's
- Karnaugh Maps
- Combinational Building Blocks
- **Timing**

Application Software	
Operating Systems	
Architecture	
Micro-architecture	
Logic	
Digital Circuits	
Analog Circuits	
Devices	
Physics	

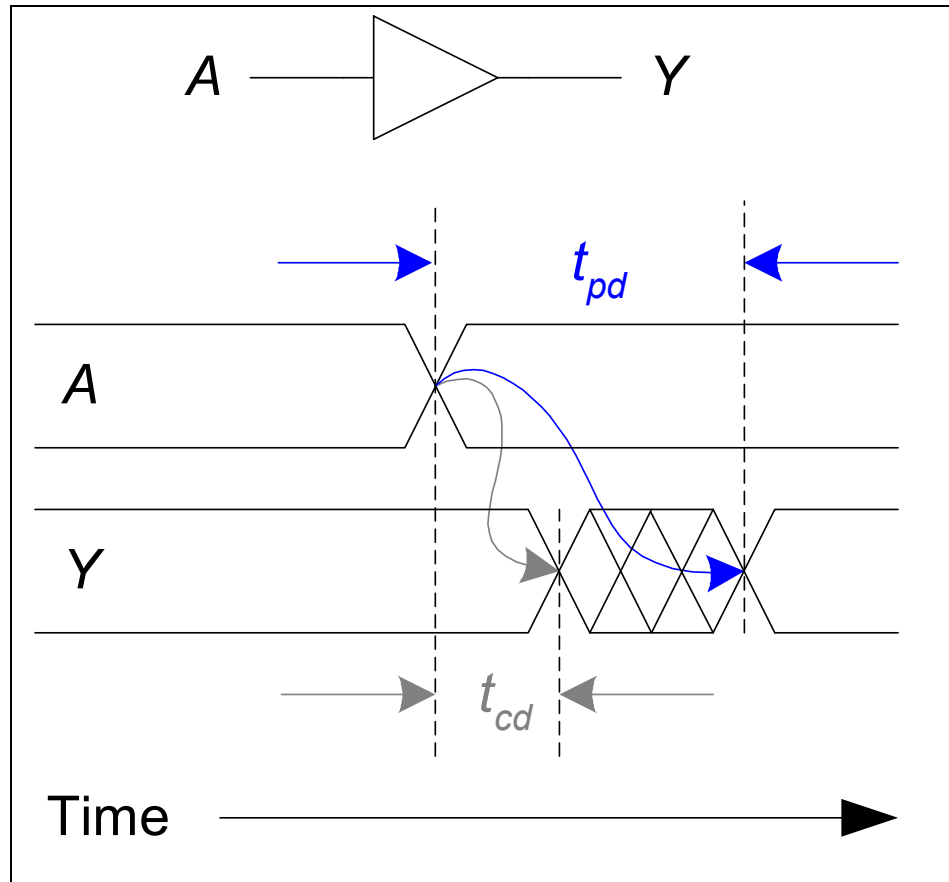
Timing

- Delay between input changing and output changing
- How to build fast circuits?



Propagation(传输) & Contamination(最小) Delay

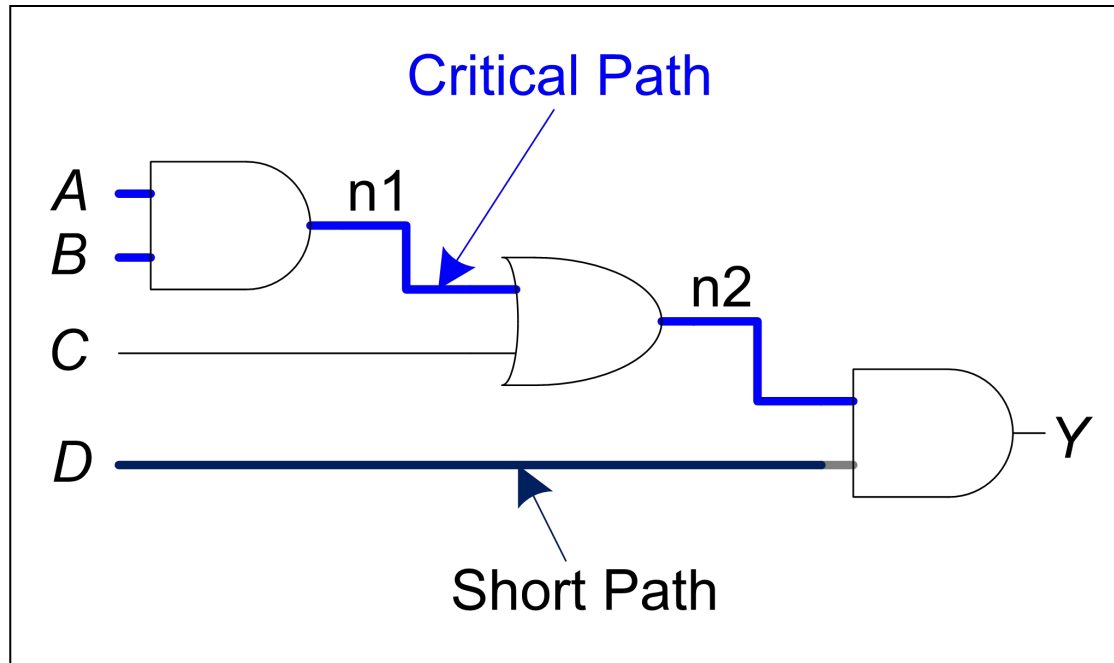
- **Propagation delay:** t_{pd} = max delay from input to output
- **Contamination delay:** t_{cd} = min delay from input to output



Propagation & Contamination Delay

- Delay is caused by
 - Capacitance and resistance in a circuit
 - Speed of light limitation
- Reasons why t_{pd} and t_{cd} may be different:
 - Different rising and falling delays
 - Multiple inputs and outputs, some of which are faster than others
 - Circuits slow down when hot and speed up when cold

Critical (Long) & Short Paths

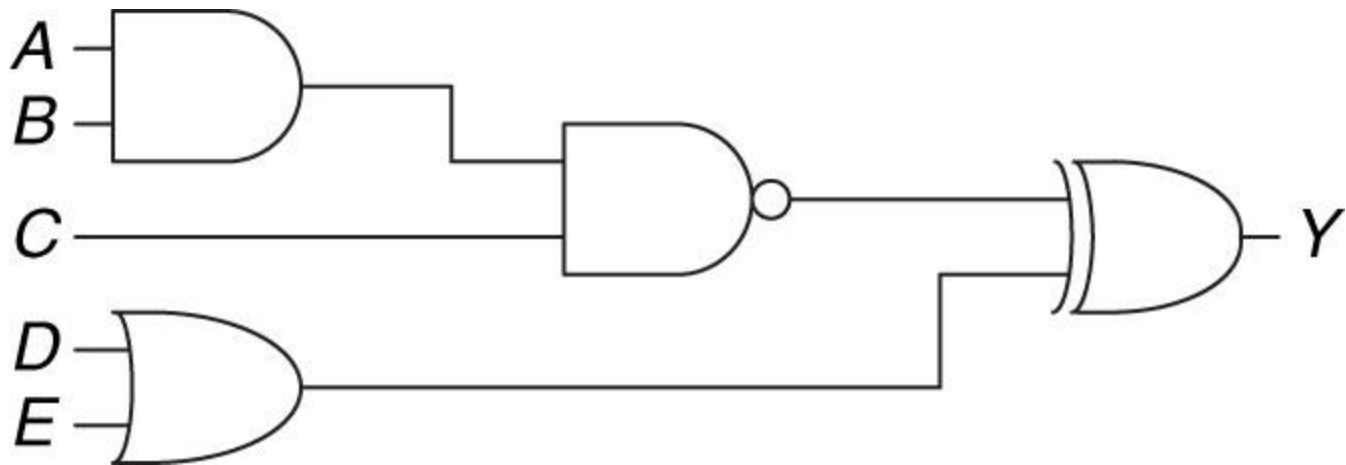


Critical (Long) Path(关键路径): $t_{pd} = 2t_{pd_AND} + t_{pd_OR}$

Short Path(最短路径): $t_{cd} = t_{cd_AND}$

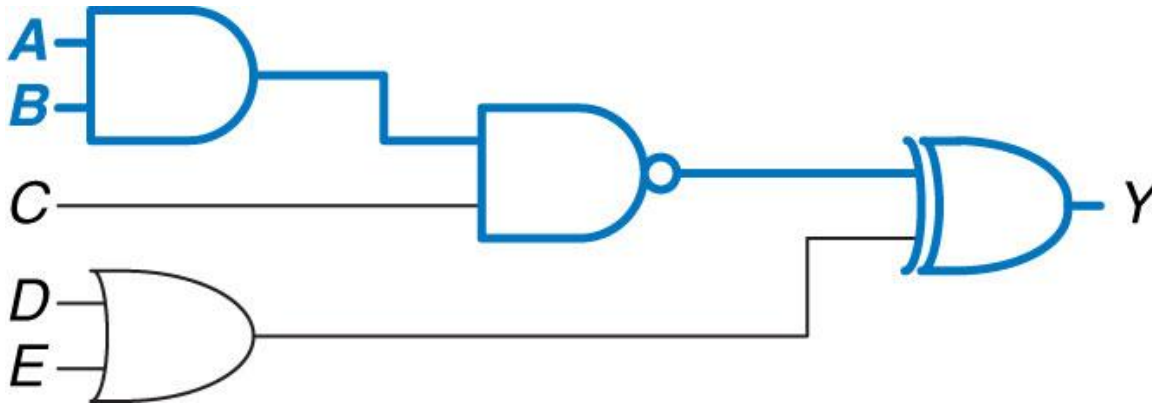
Example 2.15 FINDING DELAYS

- Ben Bitdiddle needs to find the propagation delay and contamination delay of the circuit shown in Figure. According to his data book, each gate has a propagation delay of 100 picoseconds (ps) and a contamination delay of 60 ps.

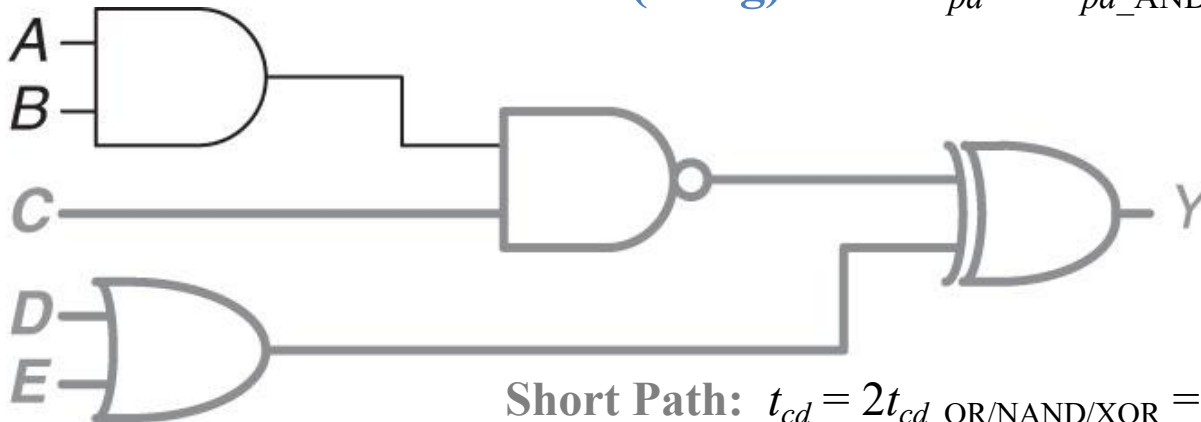


Example 2.15 FINDING DELAYS

- Solution: Ben begins by finding the critical path and the shortest path through the circuit.



Critical (Long) Path: $t_{pd} = 3t_{pd_AND/NAND/XOR} = 300\text{PS}$



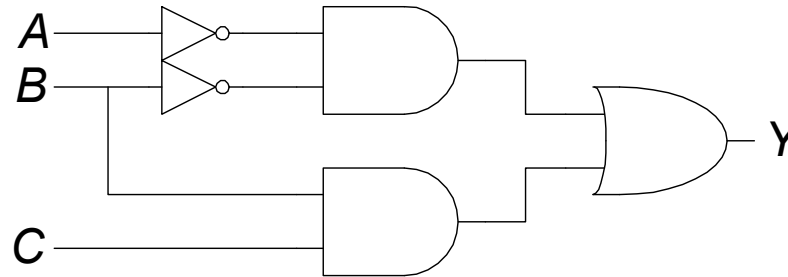
Short Path: $t_{cd} = 2t_{cd_OR/NAND/XOR} = 120\text{PS}$

Glitches

- When a single input change causes multiple output changes

Glitch Example

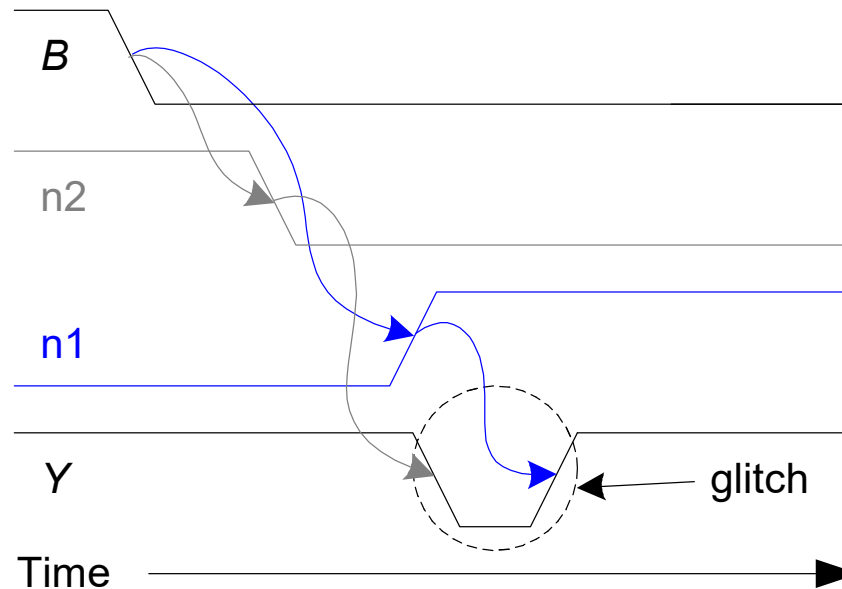
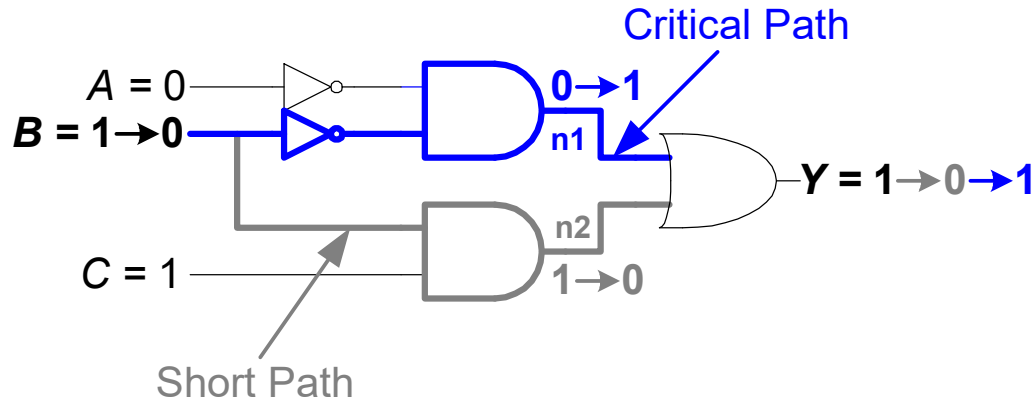
- What happens when $A = 0$, $C = 1$, B falls?



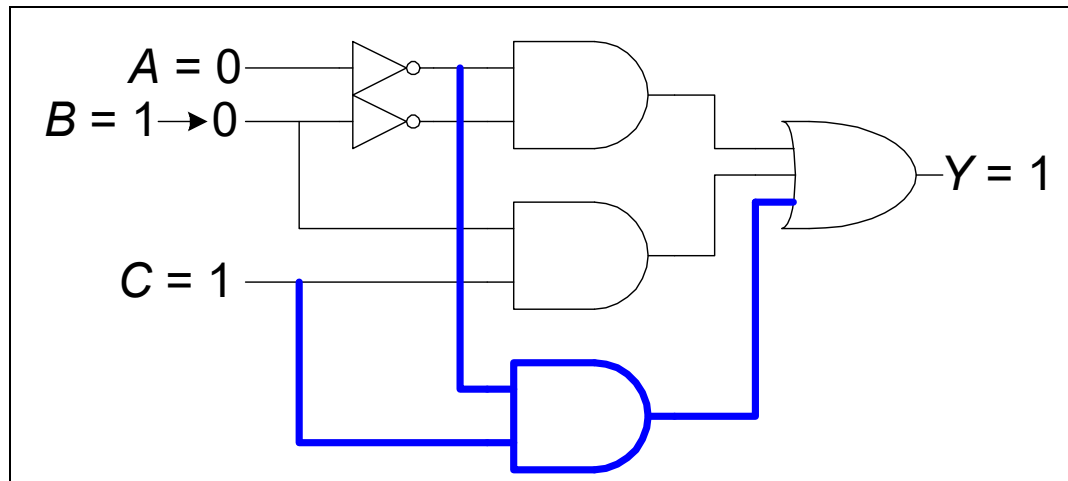
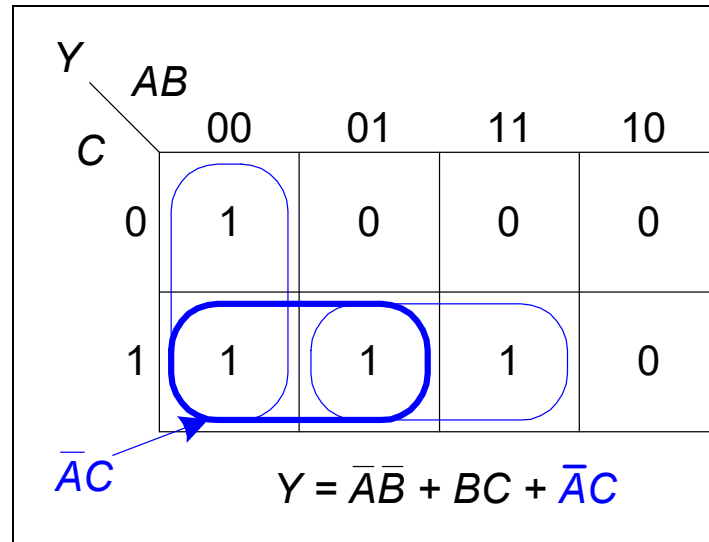
		AB			
		00	01	11	10
C	0	1	0	0	0
	1	1	1	1	0

$$Y = \bar{A}\bar{B} + BC$$

Glitch Example (cont.)



Fixing the Glitch



Why Understand Glitches?

- Can't get rid of all glitches – simultaneous transitions on multiple inputs can also cause glitches
- It's important to **recognize** a glitch: in simulations or on oscilloscope
- Glitches don't cause problems because of **synchronous design** conventions (see Chapter 3)

Additional knowledge

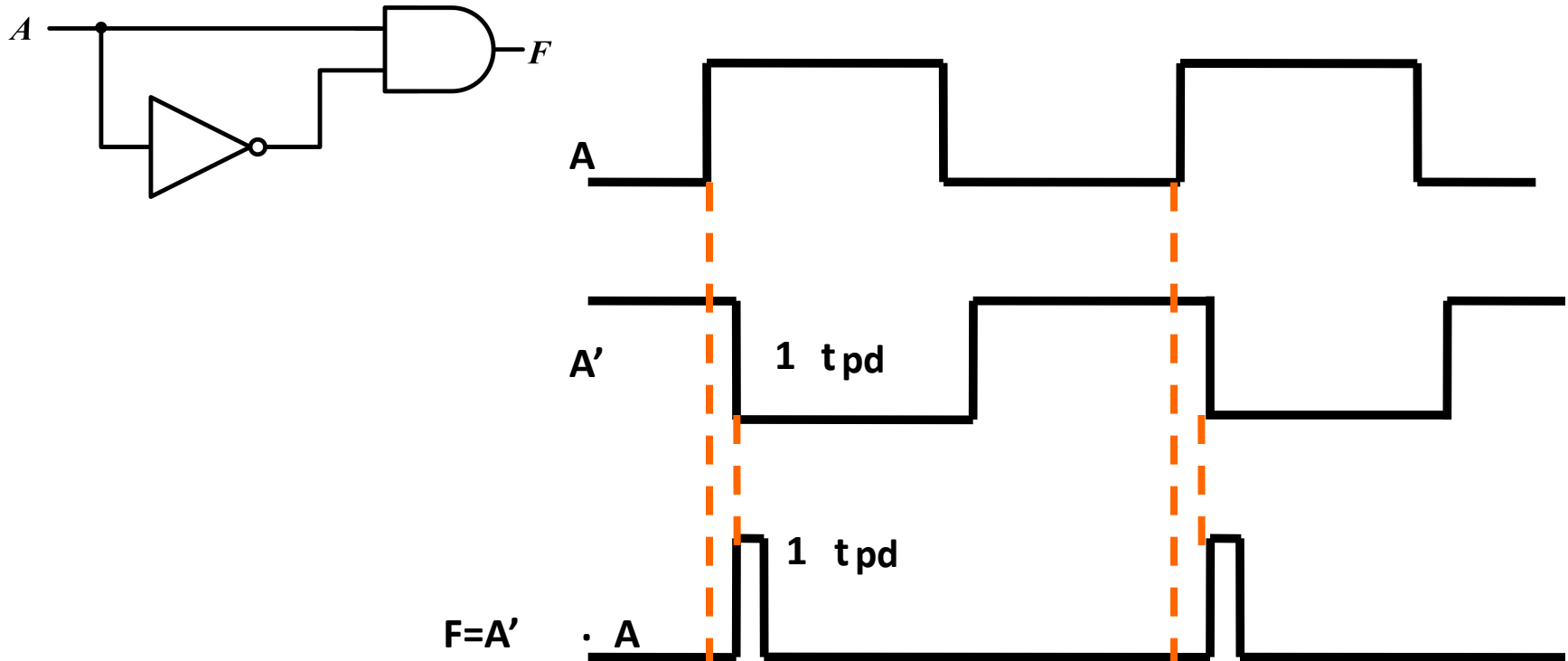
组合逻辑电路的竞争-冒险

- 由于受电路的延迟等因素的影响，使得原本正常的逻辑关系，发生了混乱，这就是产生了竞争-冒险现象。
- 一个门电路只要有二个输入信号同时向相反方向变化(即由01变为10，或者相反)，其输出端就可能产生干扰脉冲。

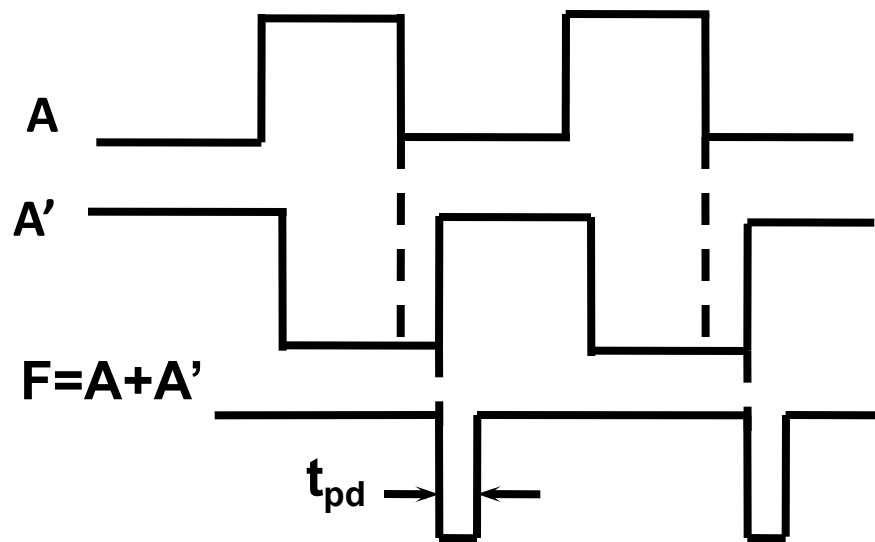
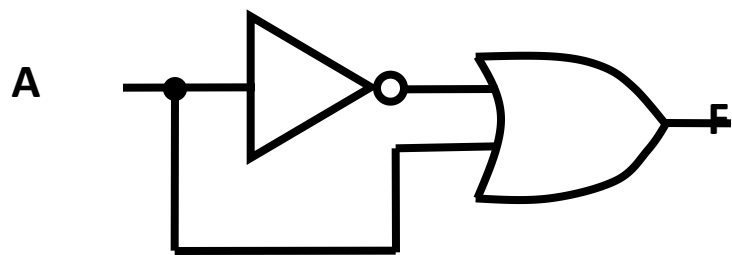
竞争-冒险的产生

- 竞争：门电路的两个输入端同时向相反的逻辑电平变化的现象称为竞争。
- 冒险：由于竞争，电路的输出端就可能出现暂短的错误输出，即出现了不应有的窄脉冲或称尖峰脉冲，这种现象称为冒险。
- 冒险的危害在于它可能使后接的时序电路产生错误操作。因而，有时要设法消除竞争冒险或尽量消除竞争冒险带来的危害。

Competition - Adventure



竞争冒险产生的正尖脉冲



(b) 波形

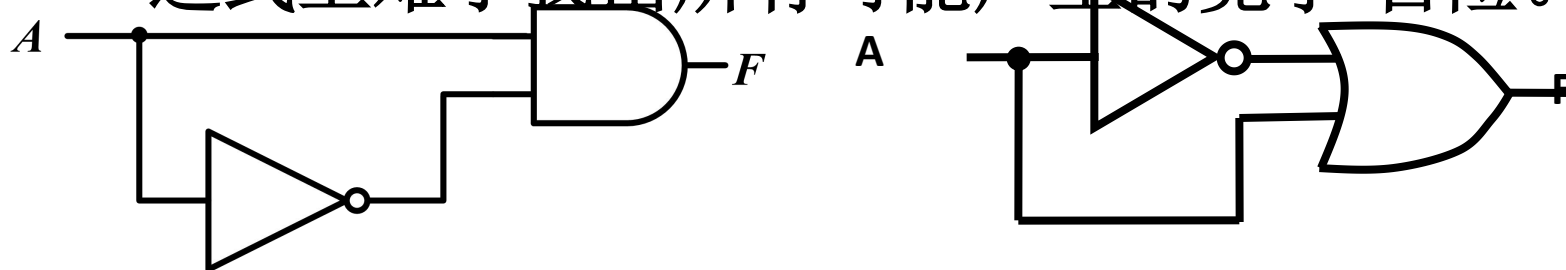
竞争产生的负尖脉冲

竞争-冒险的判断

- 1. 表达式法

- 由下面的图看出只要输出逻辑函数在一定条件下变换成 $F=A \cdot A'$ 或 $F=A+A'$ 就可能产生竞争-冒险。

- 但是当输入变量的数目较多时，从逻辑函数表达式上难于找出所有可能产生的竞争-冒险。



竞争-冒险的判断

- 2.卡诺图法

- 首先将逻辑关系用卡诺图表示，若在卡诺图画包围圈时，出现两个包围圈相切而不相交的情况，对应的逻辑电路将存在竞争-冒险。

竞争-冒险的判断

- 3.软件仿真及实验法
 - 计算机辅助分析软件也为检查复杂数字电路的竞争-冒险现象提供了有效的手段。
 - 通过采用标准化的典型参数或做一些近似，在计算机上运行数字电路的仿真程序，能够在一定程度上查出电路是否存在竞争-冒险现象。
 - 通过实验的方法可以最终确定有无竞争-冒险。

竞争-冒险的消除

- 当判定电路中存在竞争冒险可能之后，而负载又是对脉冲敏感的电路，则应该设法消除竞争冒险。方法有：
 - 修改逻辑设计
 - 输出端接滤波电容
 - 引入选通脉冲

1.修改逻辑设计

- 通过在逻辑函数表达式上加上冗余项，即可消除竞争-冒险。
- 设某逻辑电路的表达式为 $F=AB+A'C$ ，当 $B=C=1$ 时， $F=A+A'$ 会产生竞争-冒险现象。
- 若将冗余项 BC 加上之后，表达式变为 $F=AB+A'C=AB+A'C+BC$ ，即使在 $B=C=1$ 时， F 也一定等于1，因此不会产生竞争-冒险。

2.输出端接滤波电容

- 由于竞争-冒险而产生的尖峰脉冲一般很窄，通常在几十纳秒以内，所以只要在输出端并接一个几十至几百皮法的滤波电容，就可以把尖峰脉冲的幅度削弱至门电路的阈值电压以下。

3.引入选通脉冲

- 组合电路中竞争-冒险的产生一般是由于信号传输延时而引起的。为了不使竞争-冒险产生的尖脉冲得以输出，可以在电路达到稳定状态之后，加入选通脉冲，竞争-冒险产生的尖脉冲就不会输出了。